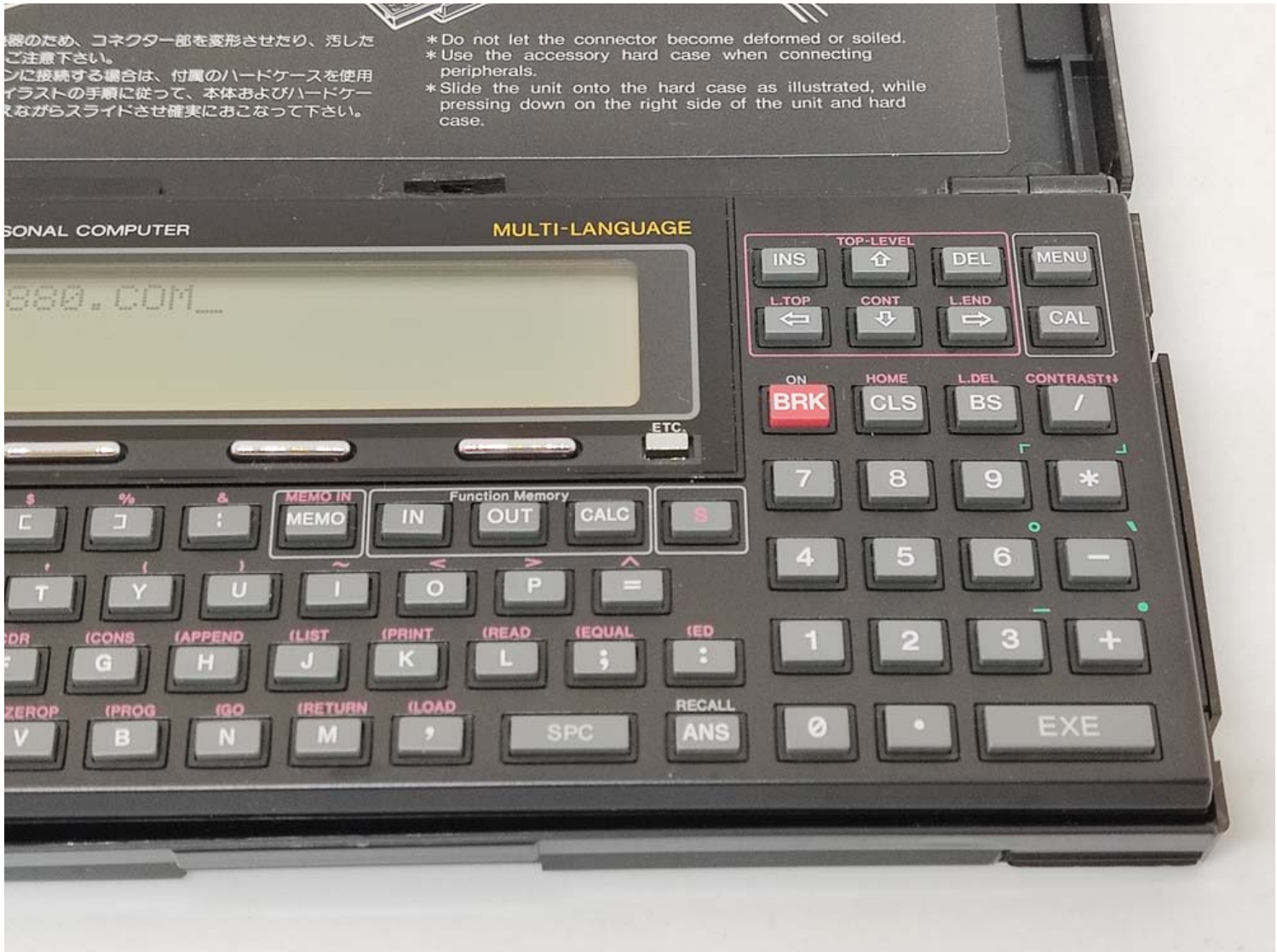
# Modelos de razonamiento

Guillermo Moncecchi
Grupo PLN - Udelar
Tercera Escuela CICADA - Tokenpalooza
Noviembre 2026


11
10


＊Do not let the connector become deformed or soiled.
＊Use the accessory hard case when connecting peripherals.
＊Slide the unit onto the hard case as illustrated, while pressing down on the right side of the unit and hard case.
SONAL COMPUTER
MULTI-LANGUAGE
880.COM_
ETC.
TOP-LEVEL
INS
DEL
MENU
L.TOP
CONT
L.END
CAL
ON
HOME
L.DEL
CONTRAST
BRK
CLS
BS
/
MEMO IN
MEMO
Function Memory
IN
OUT
CALC
S
7
8
9
*
4
5
6
−
1
2
3
+
0
.
EXE
SPC
RECALL
ANS


CASIO ROM CARD SYSTEM
LOGIC PROGRAMMING
ARTIFICIAL INTELLIGENCE
OM-51P
PROLOG

# Introducción

- Razonar, según la RAE
- Razonar, según la Wikipedia

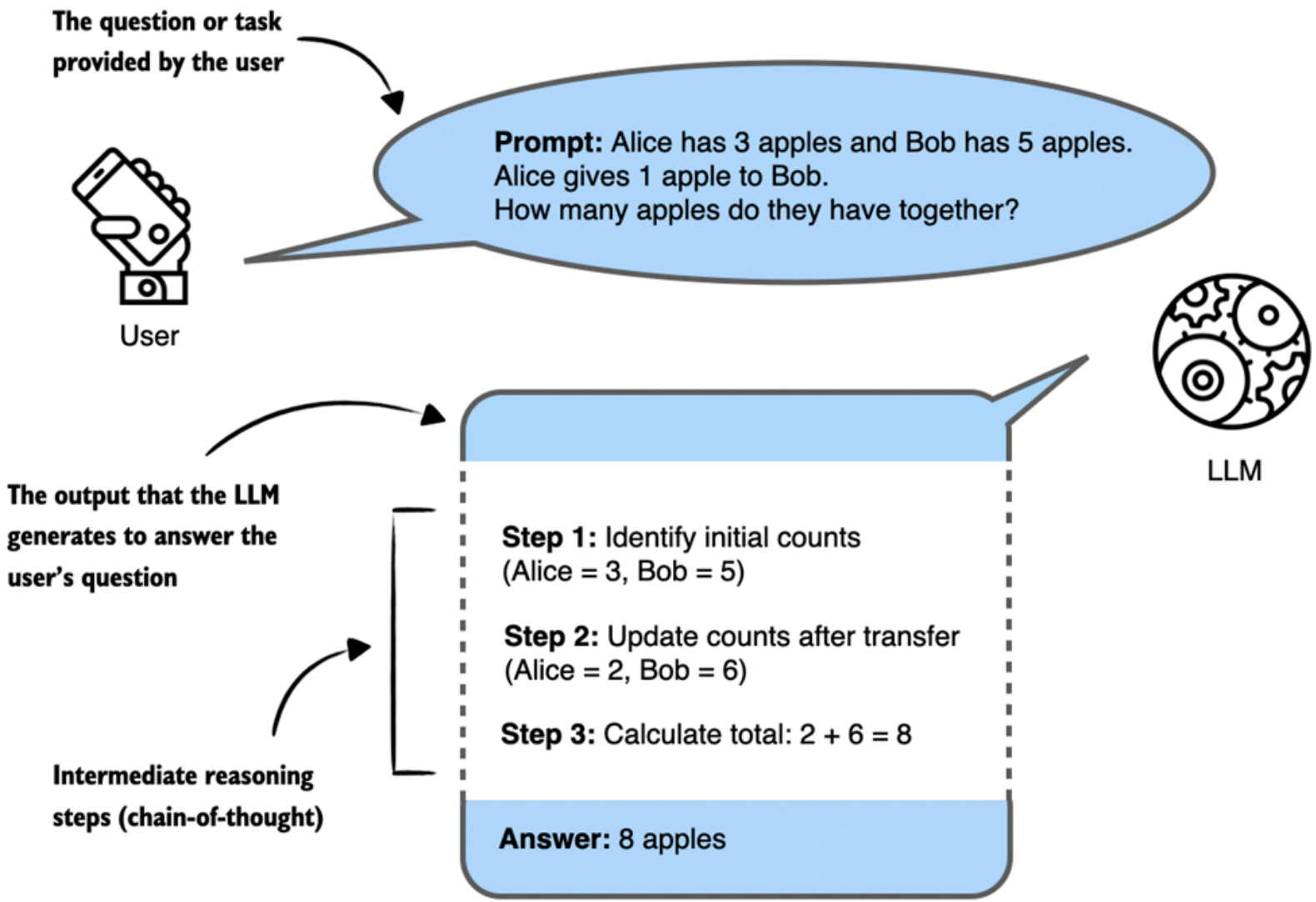
# Introducción

- Razonar, según la RAE
- Razonar, según la Wikipedia
- Cuando hablamos de "razonamiento", en el contexto de los LLMs, hablamos (o podríamos hablar) en realidad de CoT:

*A chain of thought is a series of intermediate natural language reasoning steps that lead to the final output* (Wei et al., 2022)

*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*
*Wei et al., (2022)*

# Introducción



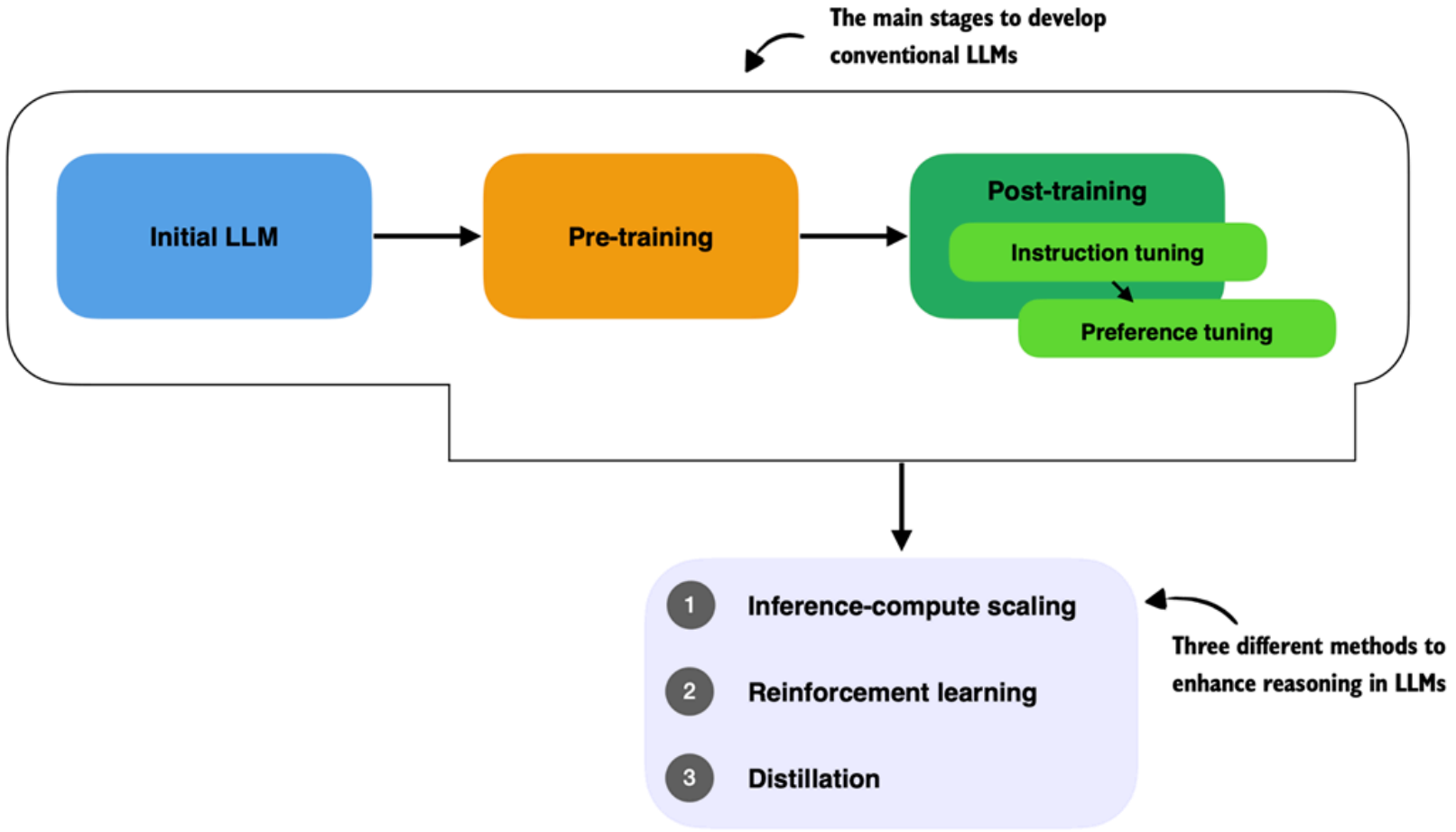
## Introducción

- Aquí no hay reglas deterministas ni paradigmas lógicos detrás.
- Los LLMs siguen siendo autorregresivos, y siguen generando, igual que antes, un token a la vez.
- Por lo tanto, los "pasos de razonamiento" no tienen por qué ser correctos.
- No hay tampoco ningún tipo de abstracción por parte del modelo

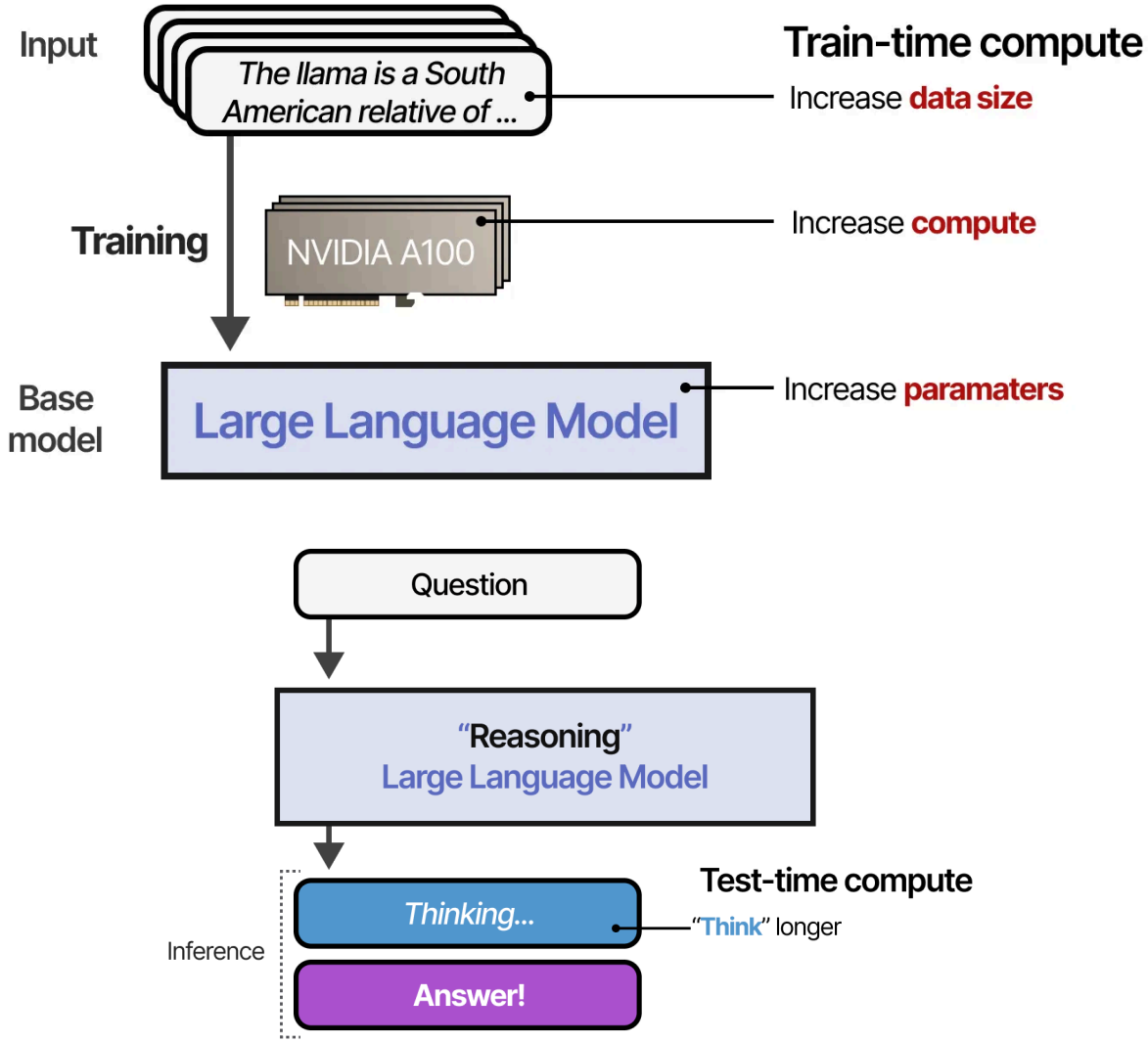
# Introducción

- 2024: Los modelos clásicos ya pueden "razonar" (2025)
- En realidad... sigue siendo reconocimiento de patrones ya vistos. No hay inferencia lógica.
- Sin embargo... funciona.
- Bueno, solamente funciona cuando:
  - Los razonamientos no son muy complejos
  - El escenario lógico es conocido
- Queremos modelos que puedan razonar "mejor". Esto es equivalente a mejorarlos para que puedan resolver problemas más complejos.

# Mejorar el razonamiento

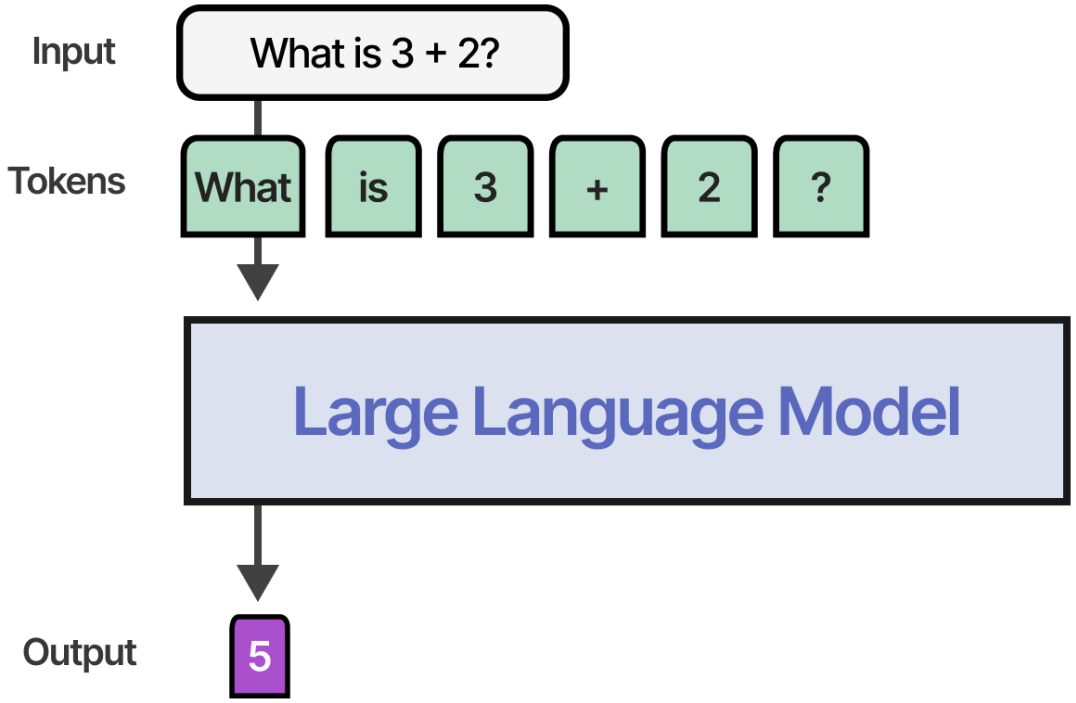


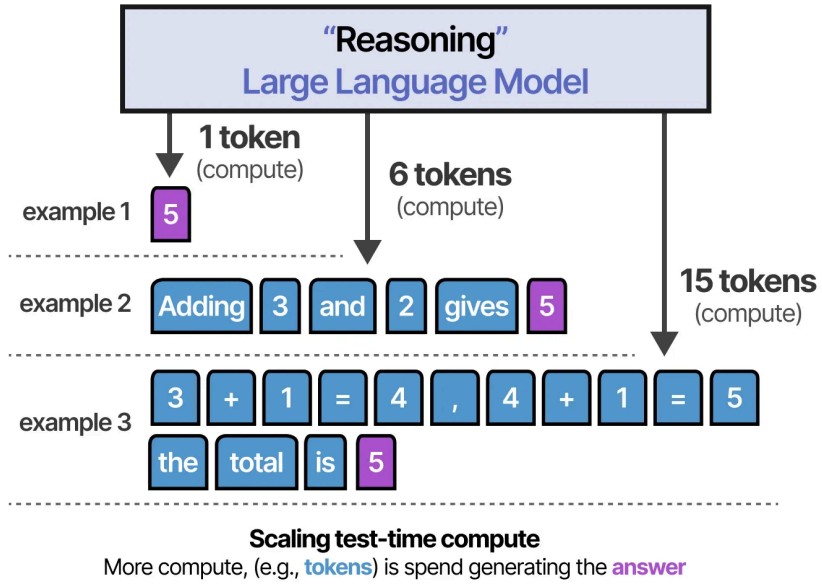
# Train-time y test-time compute



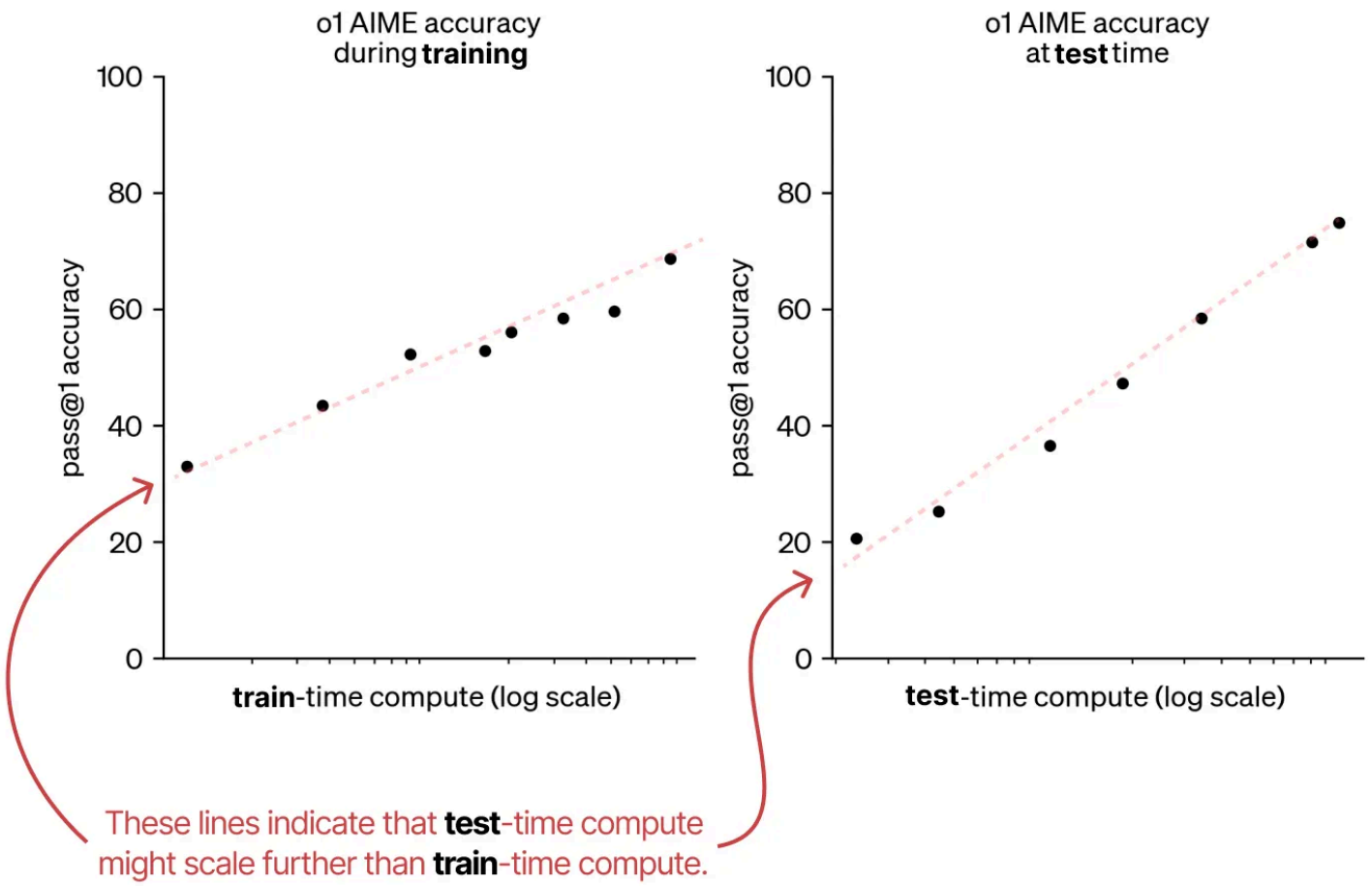
# (1) Test-time (o inference-time) compute scaling

- No modificamos los pesos
- Le damos más tiempo al generar.



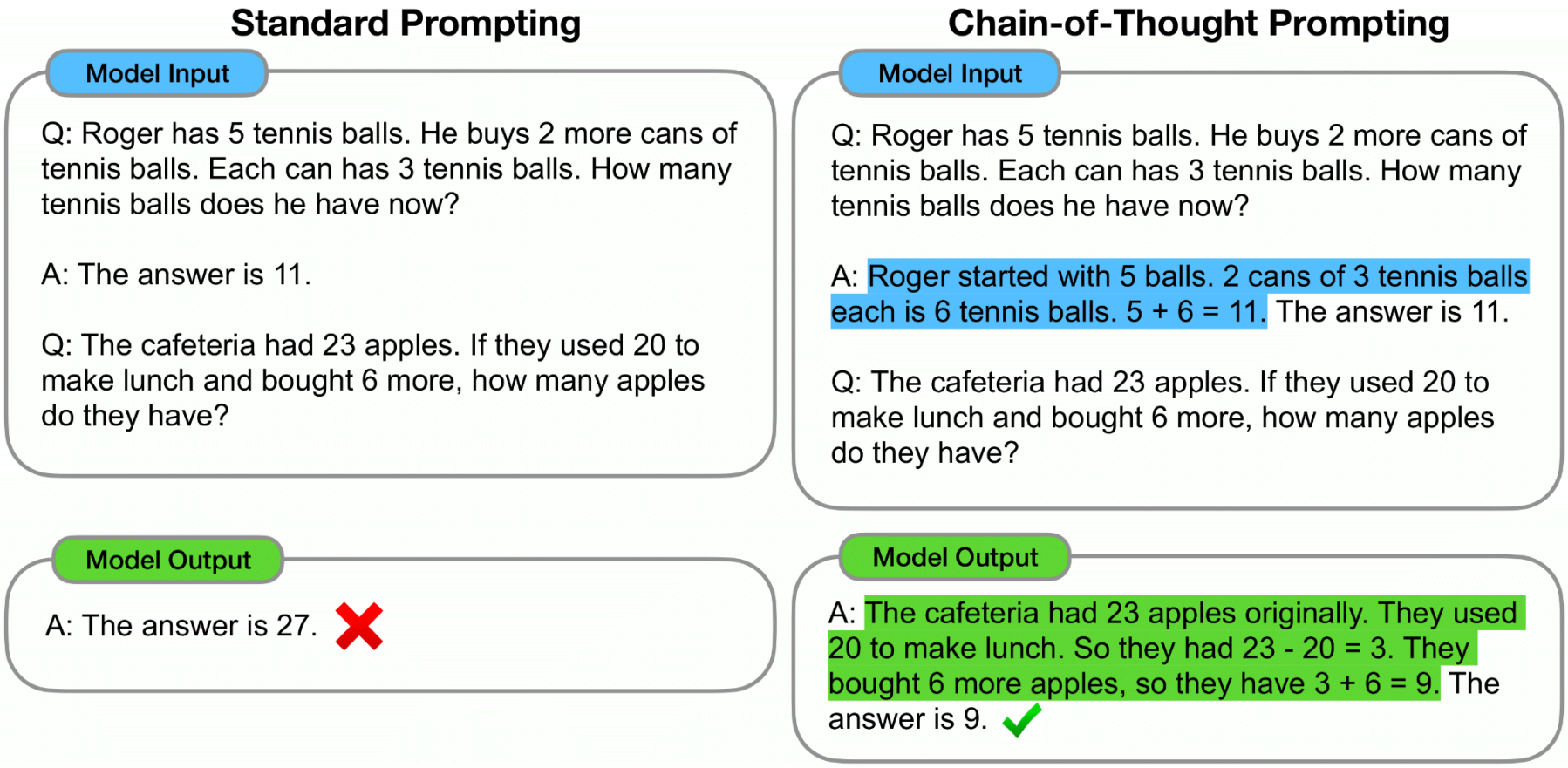



# Scaling laws...




*Learning to Reason with LLMs*
*OpenAI (2024)*

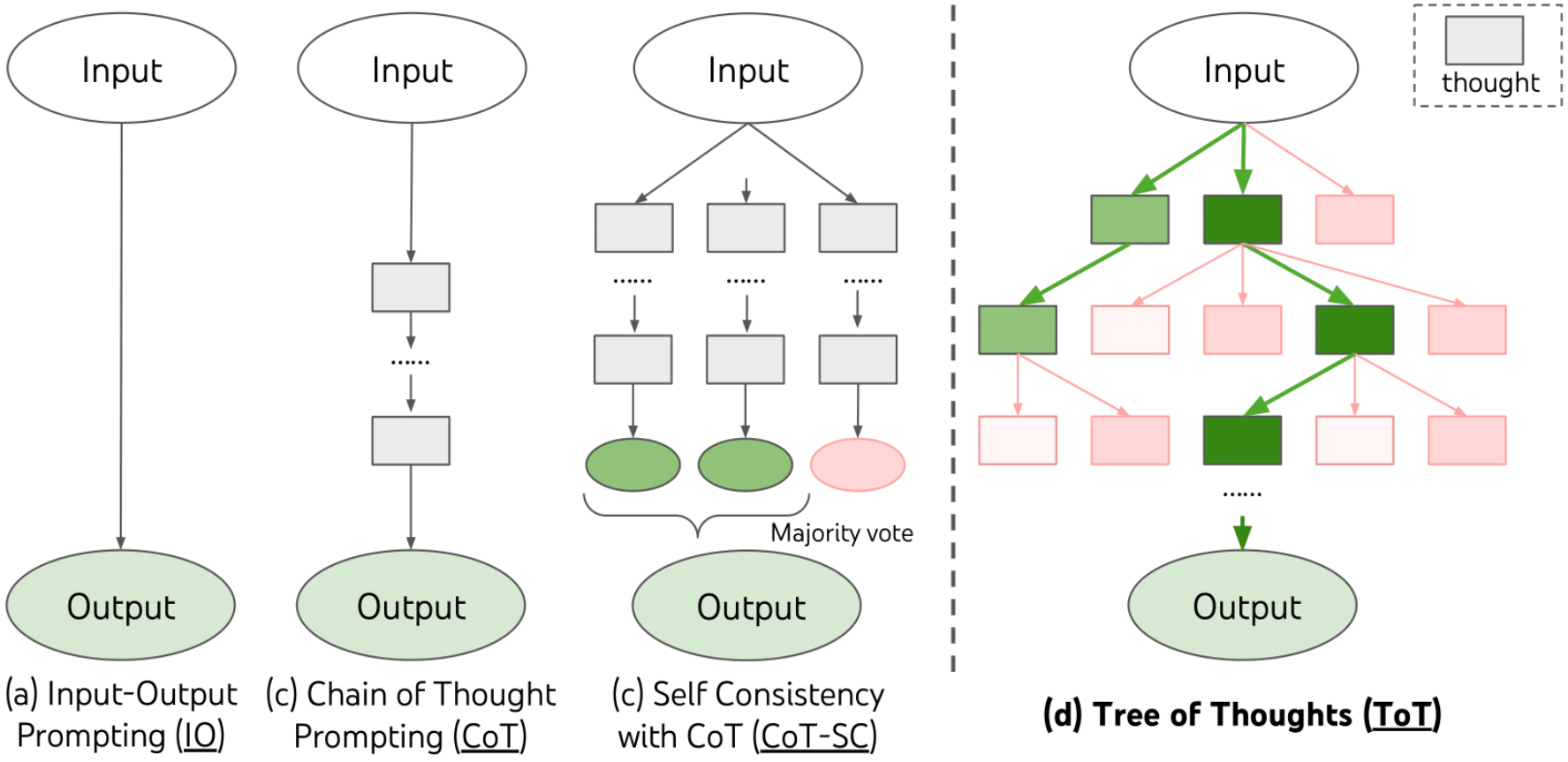
# Chain-of-Thought (CoT) Prompting




*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*

*Wei et al., (2022)*

# CoT, CoT SC, ToT

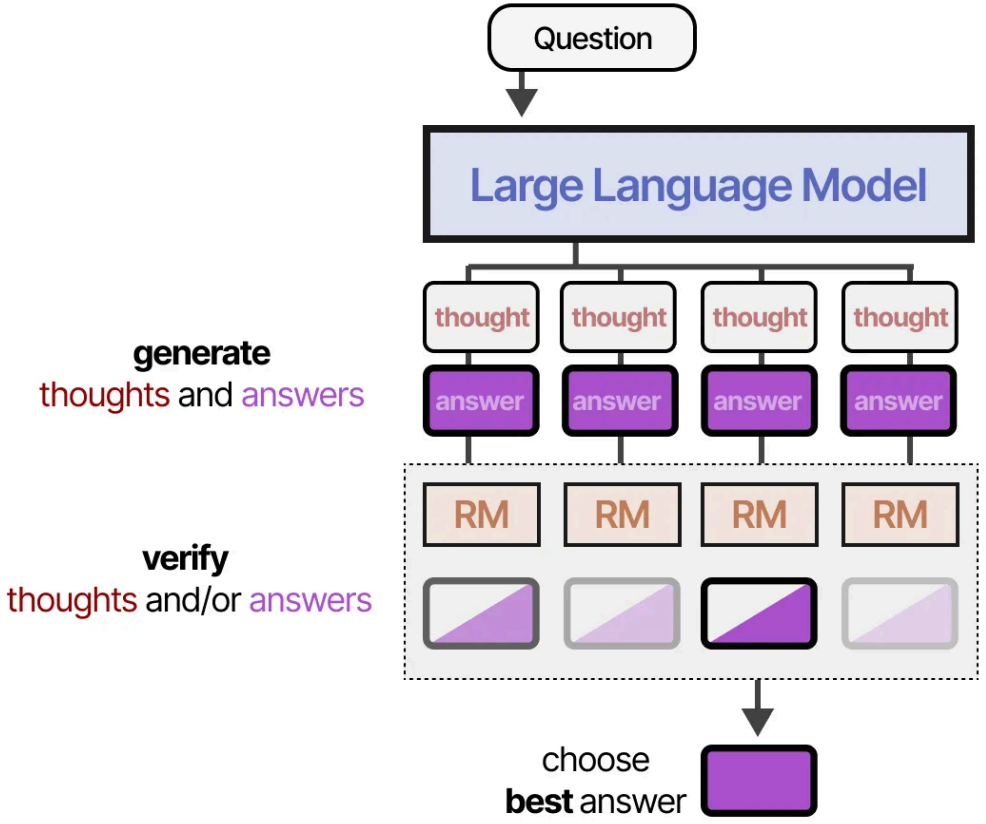



*Tree of Thoughts: Deliberate Problem Solving with Large Language Models*

*Yao et al.(2023)*

# Comparar con verificadores

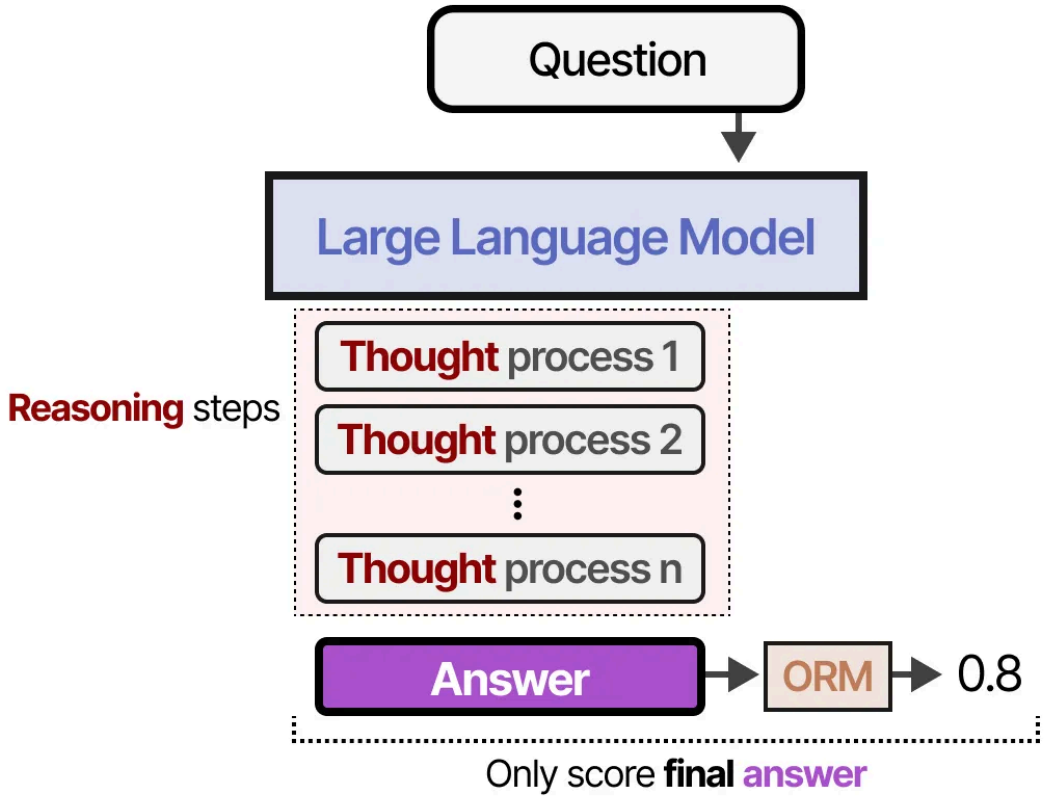
Generamos varias respuestas y elegimos la "mejor"




*Scaling LLM test-time compute optimally can be more effective thn scaling model parameters*
*Snell et al.(2024)*

# Modelos de recompensa (Reward Models)

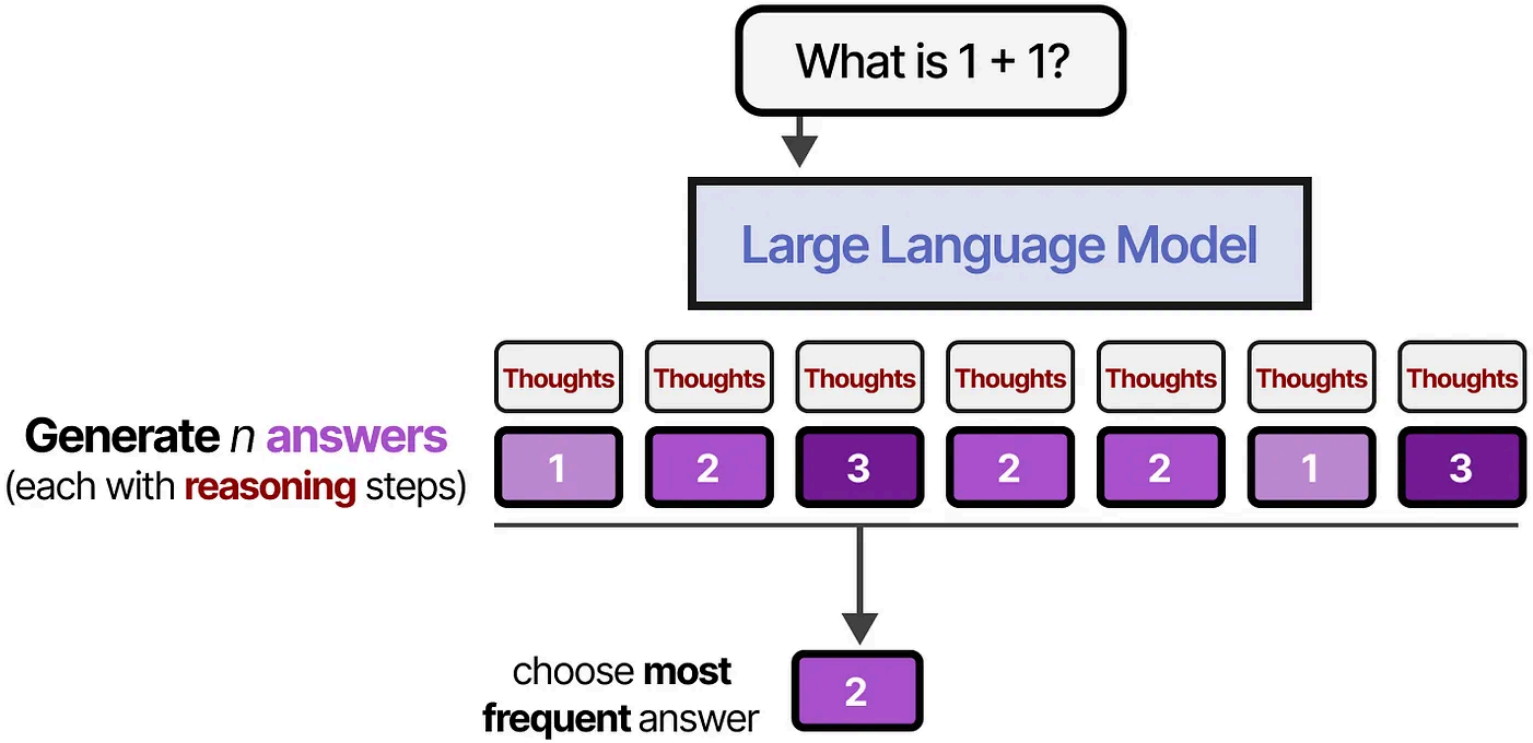
- Outcome reward model: un modelo entrenado para verificar



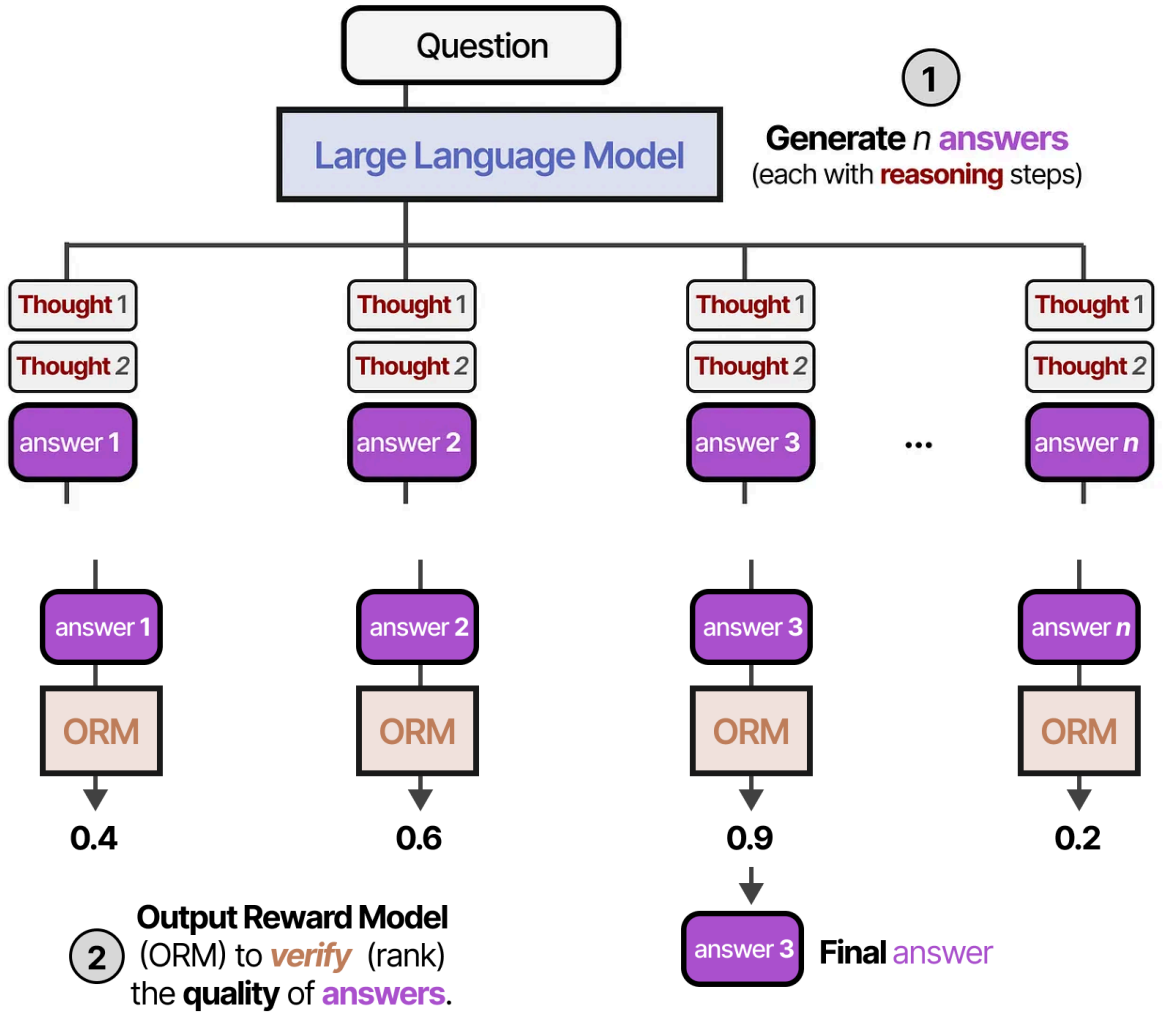

*Training Verifiers to Solve Math Word Problems*
*Cobbe et al.(2021)*

# Voto por mayoría

- Generamos varias respuestas y las evaluamos con un RM



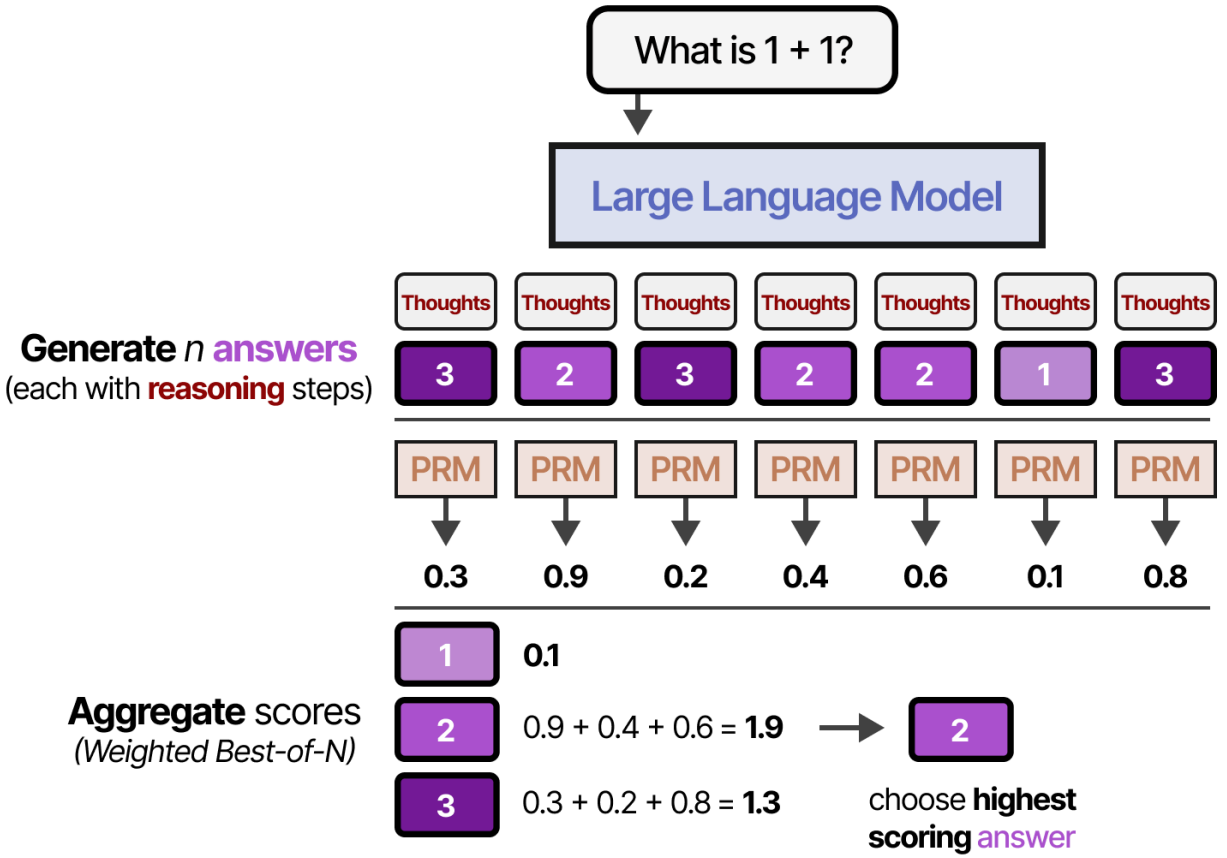
# Best-of-n samples





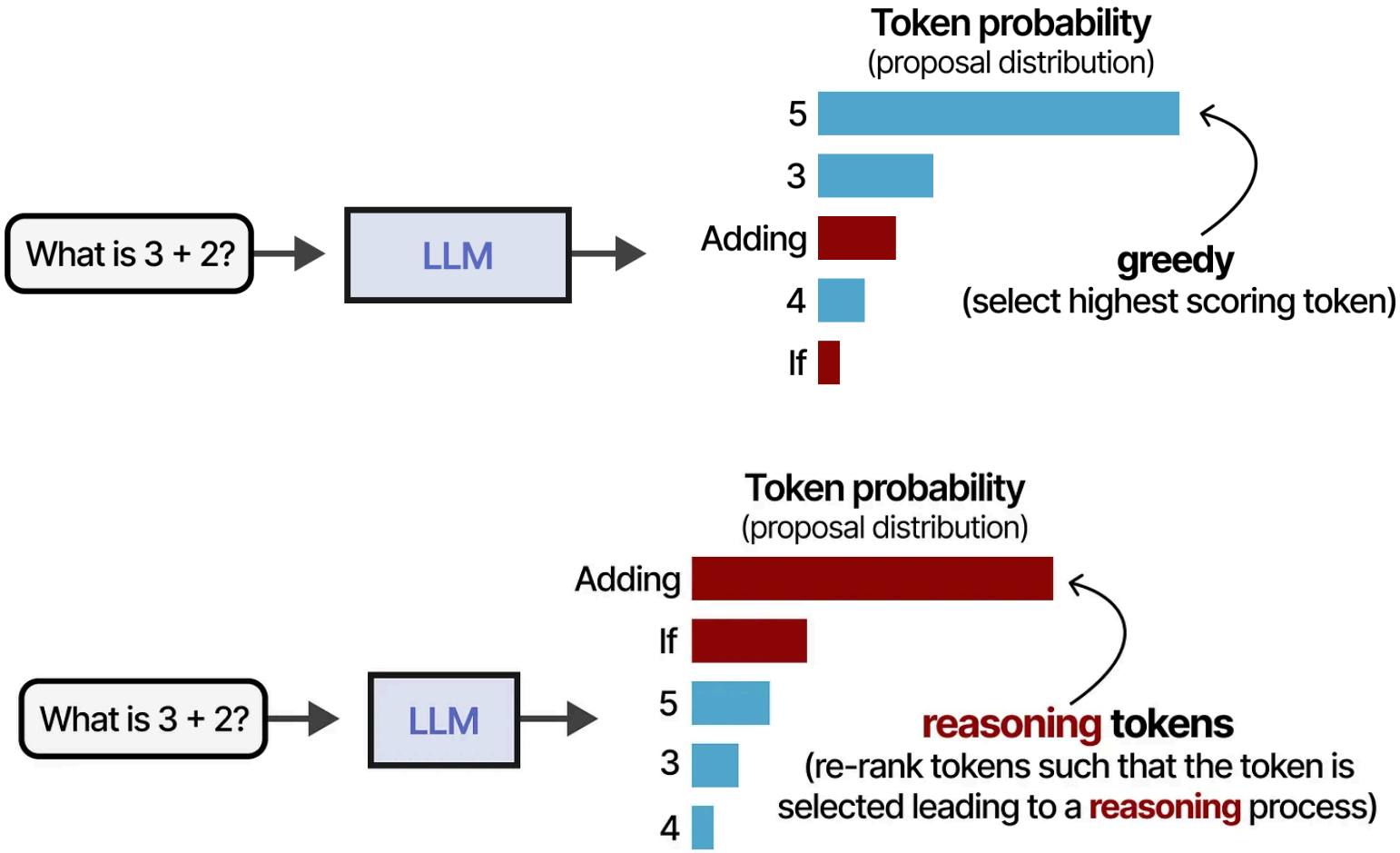

*Learning to reason with LLMs (OpenAi, 2025)*

# Best-of-n samples

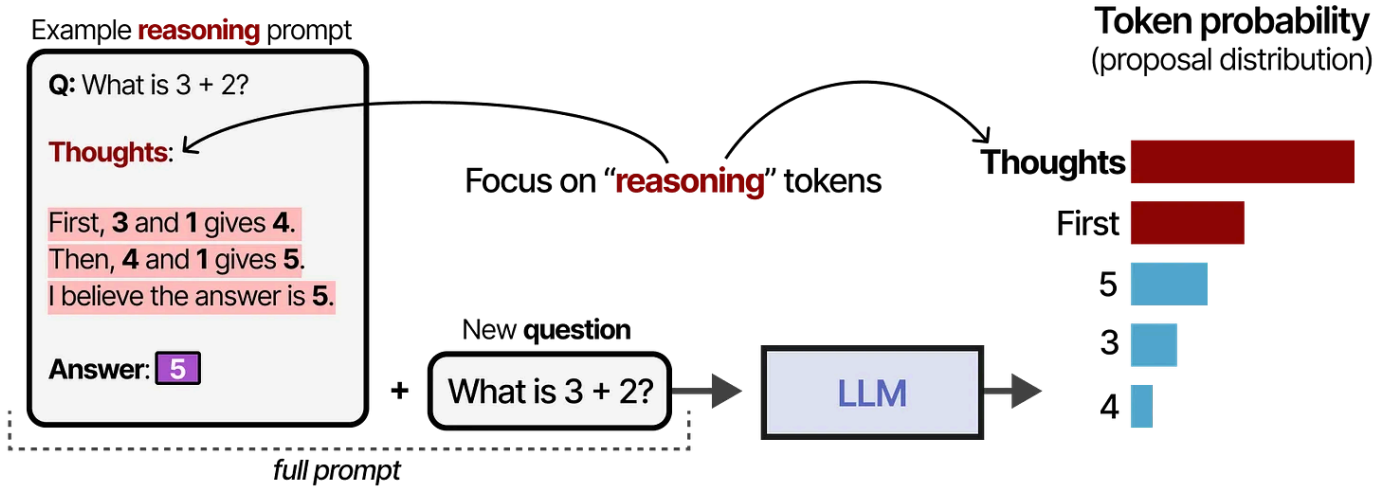


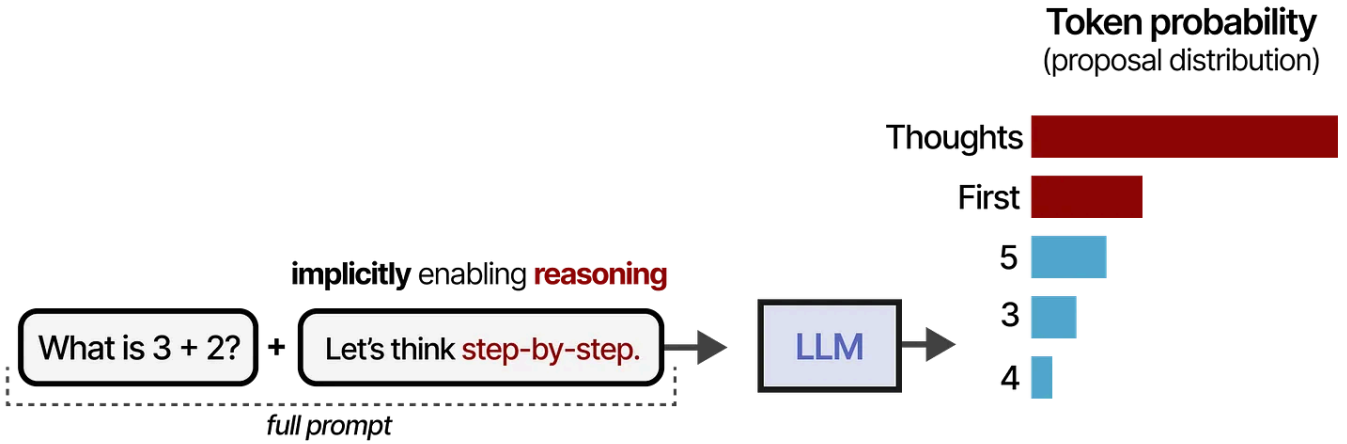

*Learning to reason with LLMs (OpenAi, 2025)*

# Alternativa: Modificar la distribución



# Modifying proposal distribution: Prompting



## (2) Reinforcement Learning: DeepSeek-R1 Zero

- Base LLM: DeepSeek-V3-Base
- Solamente RL (sin supervised fine-tuning) para incorporar razonamiento
- Accuracy rewards (evaluar la respuesta)
- Format rewards
- RL: GRPO: modifica las elecciones (tokens, pasos de razonamiento) que llevaron a una respuesta correcta/incorrecta para que sean más/menos probables.

*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement DeepSeek AI (2025)*

# Reinforcement Learning y LLMs

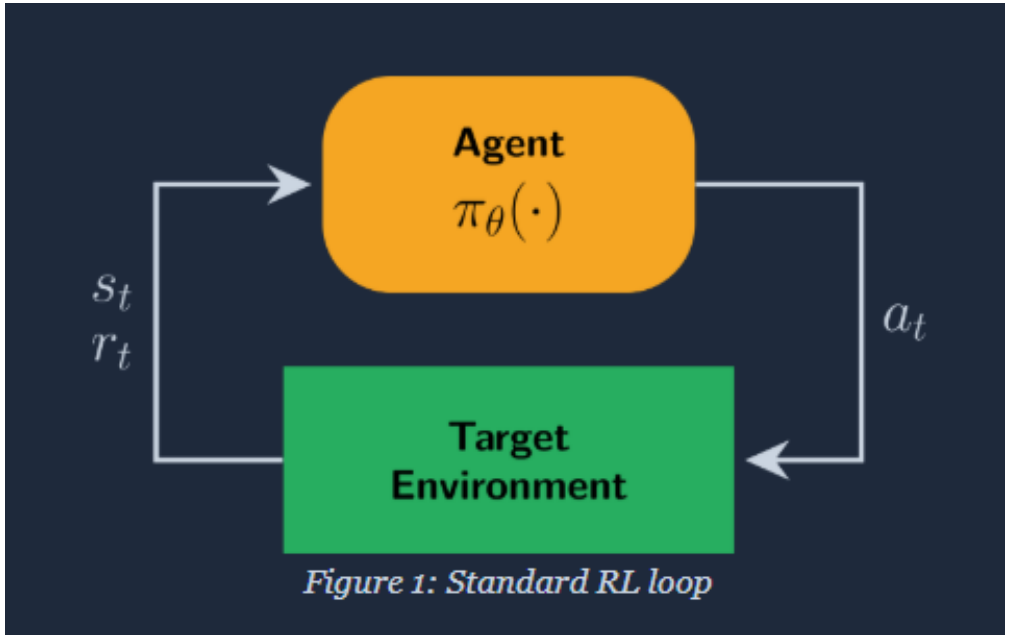
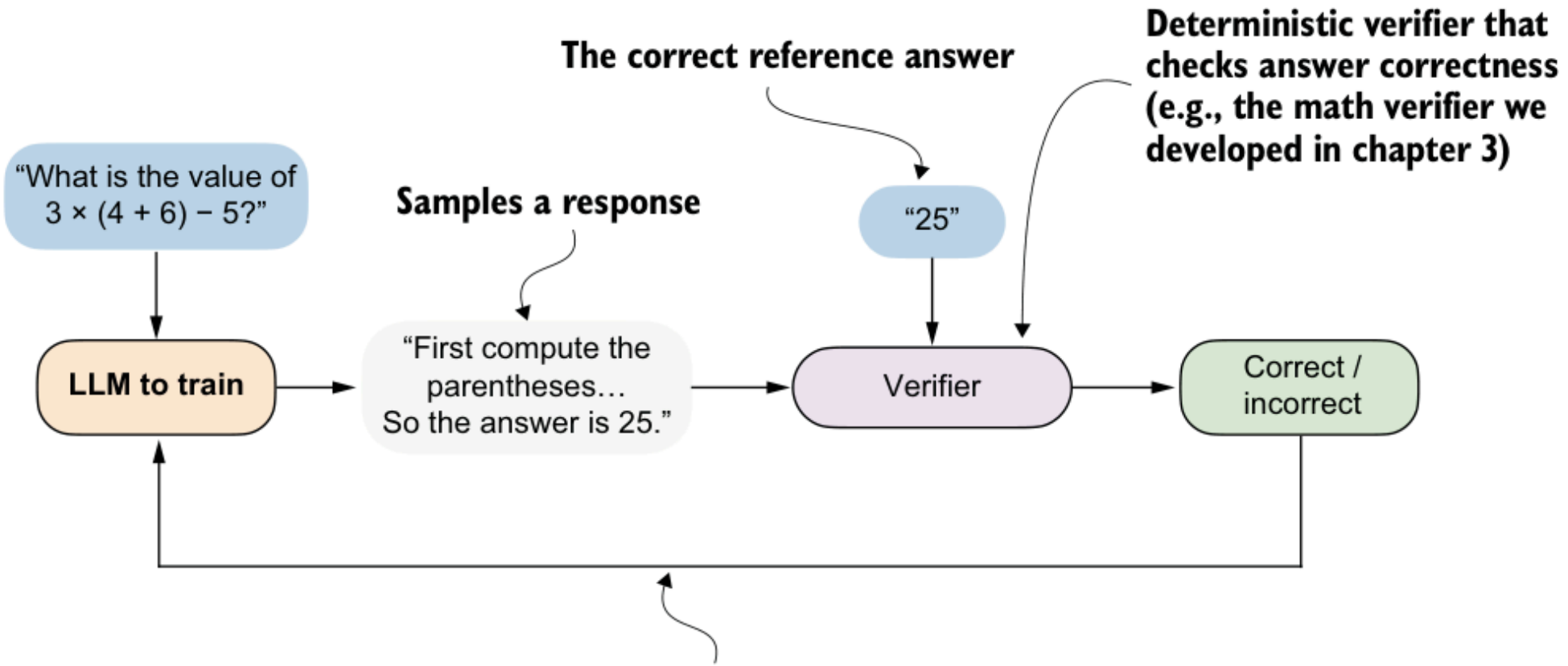


*Figure 1: Standard RL loop*

# RL estándar vs. RL con LLMs

| Componente | RL estándar | RL con LLMs |
|---|---|---|
| Política $\pi$ | Se aprende desde cero (inicialización aleatoria) | El propio LLM $\pi_\theta$, partiendo de un modelo preentrenado |
| Estado $s_t$ | Estado del entorno | El prompt $x$ (muestreado de un dataset) |
| Acción $a_t$ | Una acción del entorno | La respuesta completa $y \sim \pi_\theta(\cdot \mid x)$ (secuencia de tokens) |
| Transición $p(s_{t+1} \mid s_t, a_t)$ | Dinámica del entorno | No hay: la respuesta no define el próximo prompt |
| Recompensa | Función del entorno $r(s_t, a_t)$ | Modelo de recompensa aprendido $r_\phi(x, y)$ o verificador (RLVR) |
| Granularidad | En cada paso | Una sola recompensa para toda la respuesta |
| Horizonte y descuento | Episodios de varios pasos ($T > 1$), $\gamma < 1$ | Un único paso ($T = 1$), sin descuento ($\gamma = 1$) |

# Reinforcement Learning with Verifiable Rewards



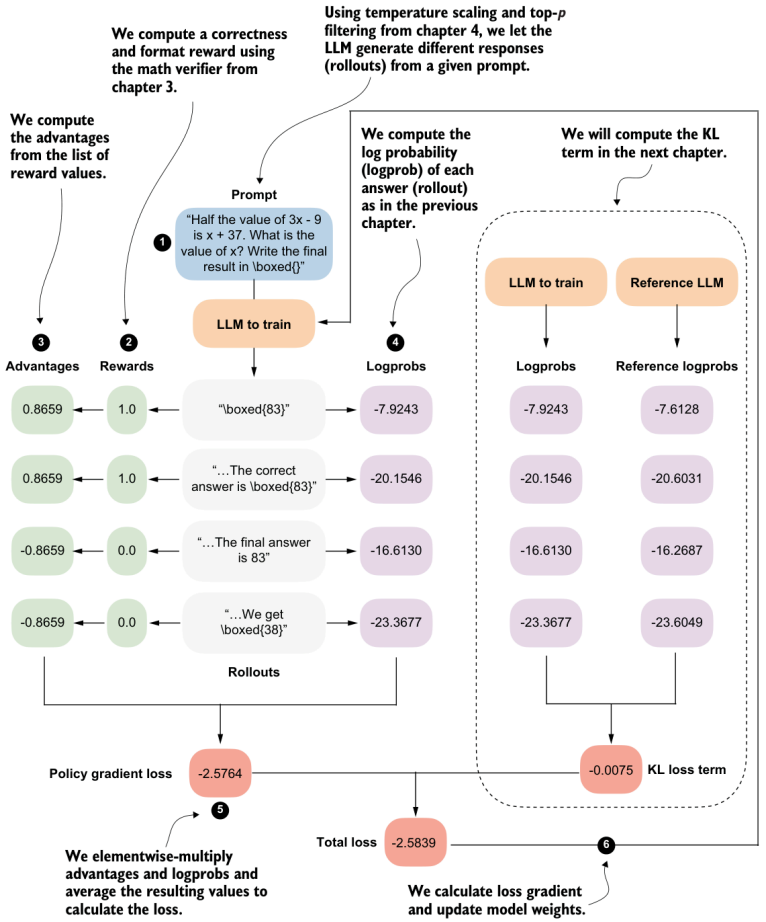
## Reinforcement Learning with Verifiable Rewards

- Los rewards se calculan de forma determinista
- DeepSeek-R1 (2025)
  - Problemas matemáticos
  - Problemas de código

## RLVR con GRPO

- Necesitamos un algoritmo de optimización de la política (política = LLM) (optimizar = ajustar los pesos y entrenar)
- The State of Reinforcement Learning for LLM Reasoning

# RLVR con GRPO - Big Picture



## Disgresión: qué es $-log(x)$

Supongamos que tenemos un evento con probabilidad $p$ de ocurrir, y queremos que la información (o la sorpresa, o la incertidumbre) de ese evento cumpla ciertas propiedades:

1. $I(p)$ decrece monótonamente con p. Es decir que si p crece, $I(p)$ decrece. Esto tiente sentido: cuanto más improbable es un evento, más nos sorprende que suceda, y tenemos más incertidumbre sobre su ocurrencia. Acertar a rojo en la ruleta no nos parece tan raro como acertar número.
2. $I(1) = 0$. Si el evento ocurre siempre, no nos da ninguna información, no disminuye en nada nuestra sorpresa.
3. $I(p_1 \cdot p_2) = I(p_1) + I(p_2)$ si ambos eventos son independientes, la sorpresa de ver ambos eventos es la suma de las sorpresas de ver cada uno por separado.
4. $I(p)$ es doblemente continua y diferenciable.

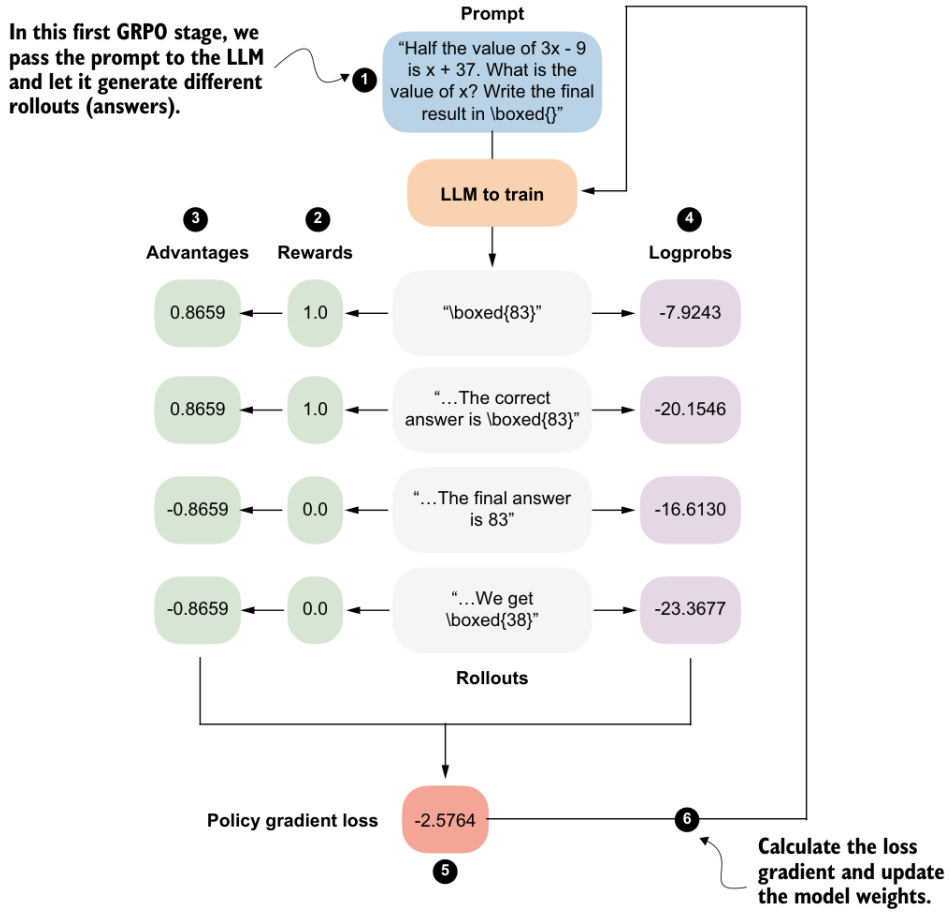
## Disgresión: qué es $-log(x)$

- Shannon encontró que una función que cumplía esto era $log(1/p) = -log(p)$ (y, de hecho, es la única, a menos de una constante $k$ que lo multiplique). Entonces, cada vez que veamos algo como $-log(p)$, recordemos primero que es lo mismo que $log(1/p)$ y que eso es una medida de la información que nos da el evento.
- Si una probabilidad es pequeña... $-log(p)$ tiende a $+\infty$
- Si la probabilidad es 1... $-log(p)$ es 0 (no hay sorpresa)
- Generalmente las funciones de pérdida quieren penalizar modelos que "se equivocan con certeza"

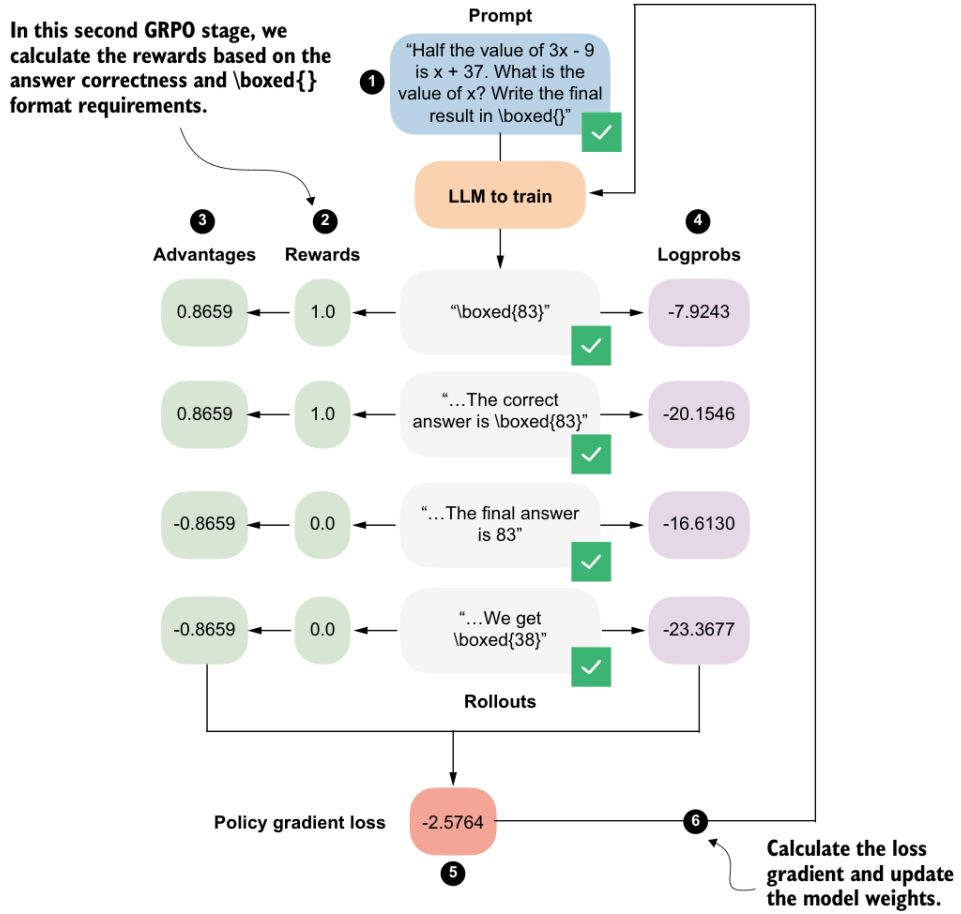
*A Mathematical Theory of Communication (Shannon, 1948)*



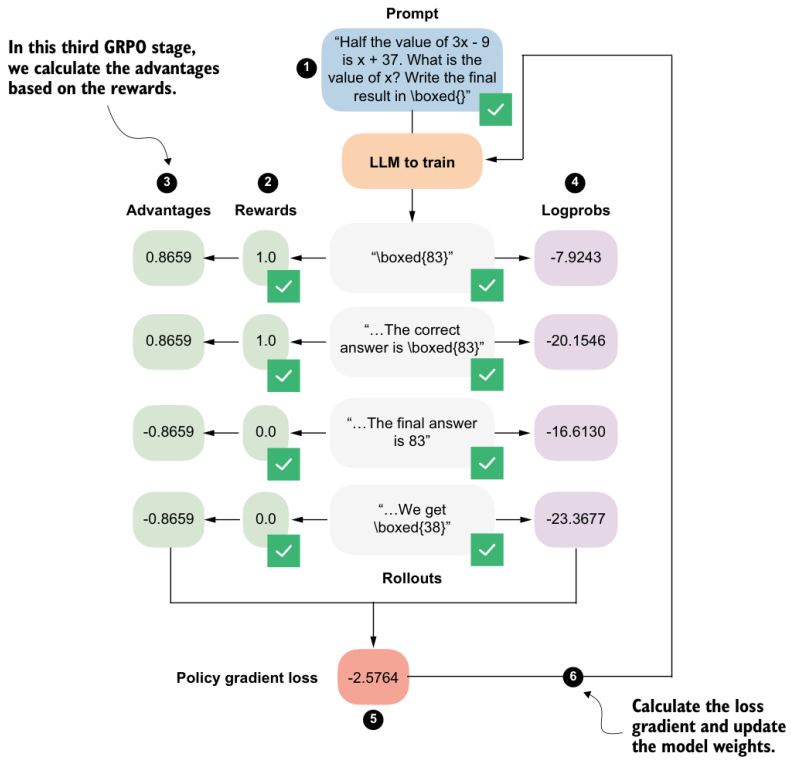
# RLVR con GRPO - Generar los rollouts



# RLVR con GRPO - Calcular las recompensas

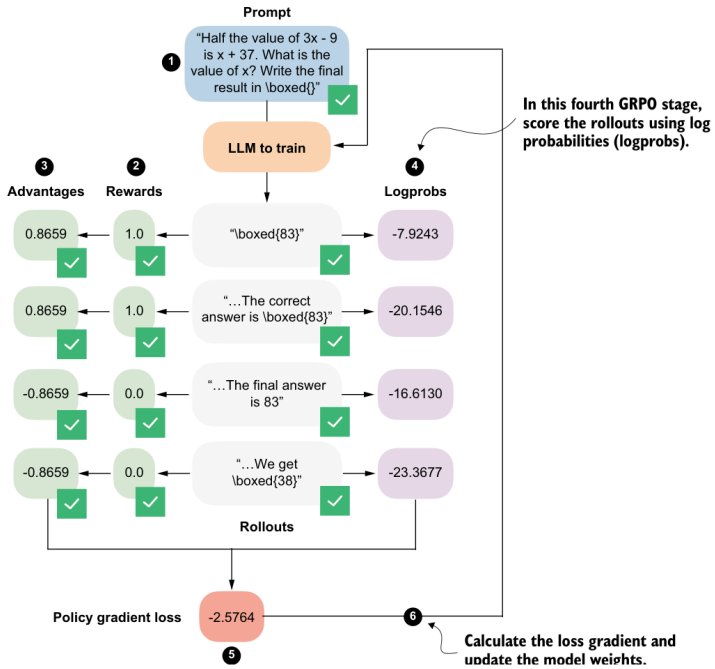


# RLVR con GRPO - Generar la señal para el aprendizaje



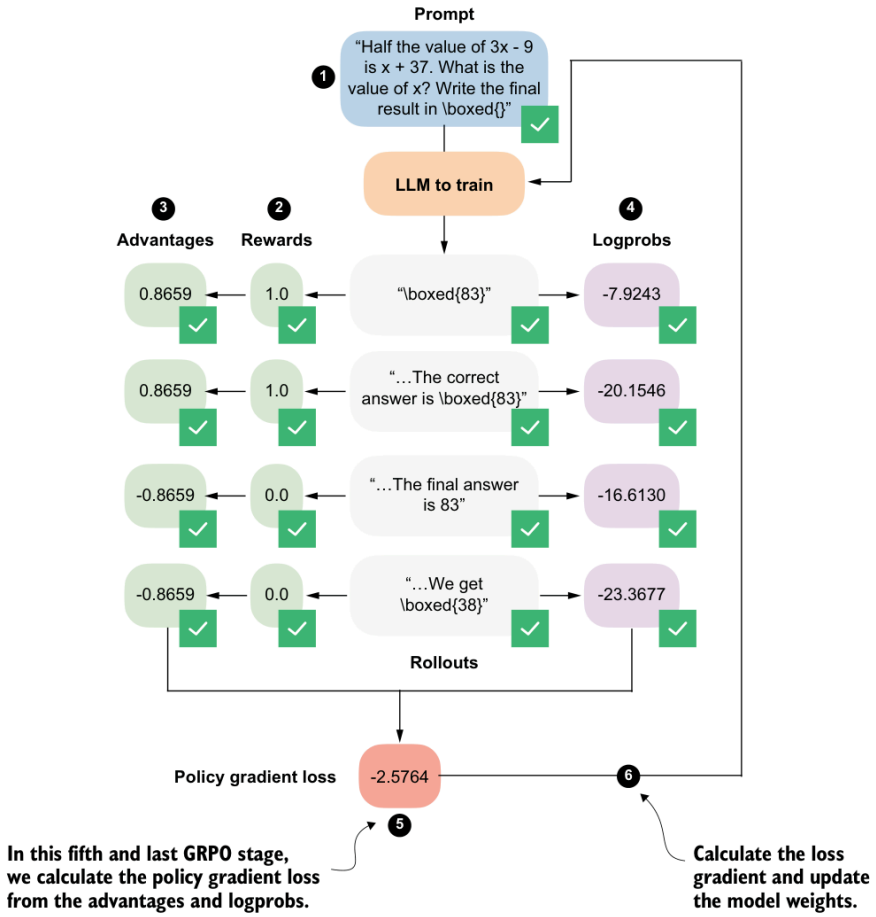

$$\text{advantages}_i = (r_i - \mu_r)/(\sigma_r + \epsilon)$$

# RLVR con GRPO - Generar la señal para el aprendizaje




Logprobs $\sum_{t=1}^{T} \log p_w(y_t \mid y_{<t}, x)$

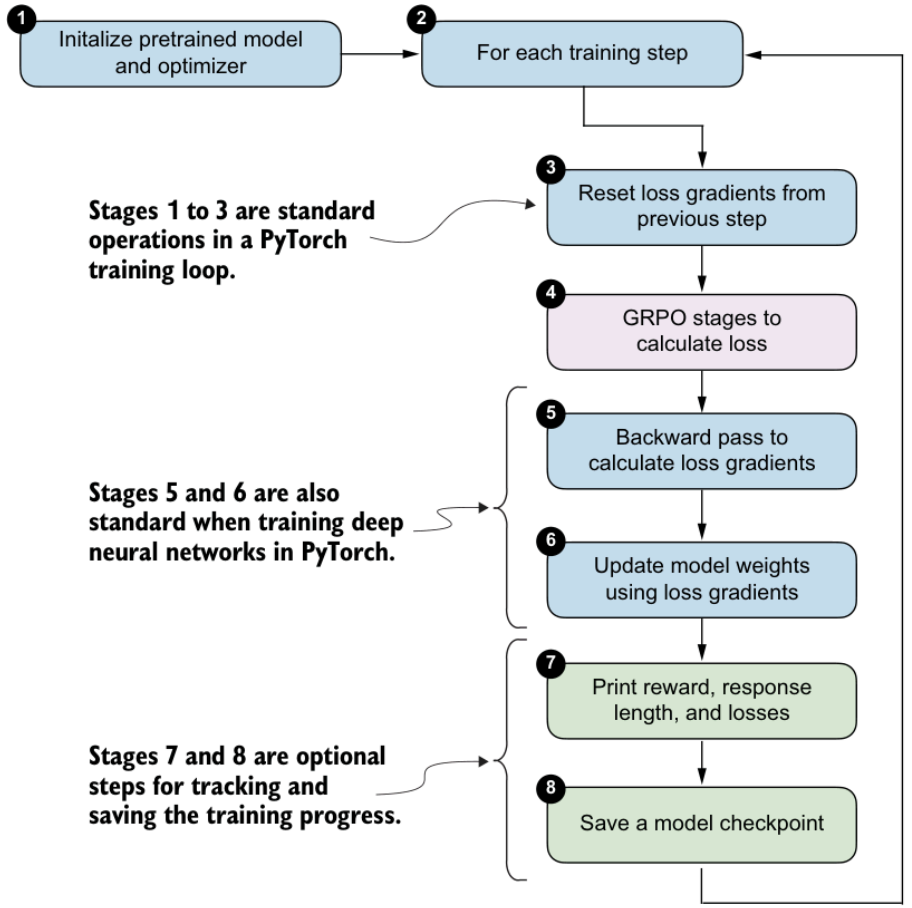
# RLVR con GRPO - Calcular la pérdida



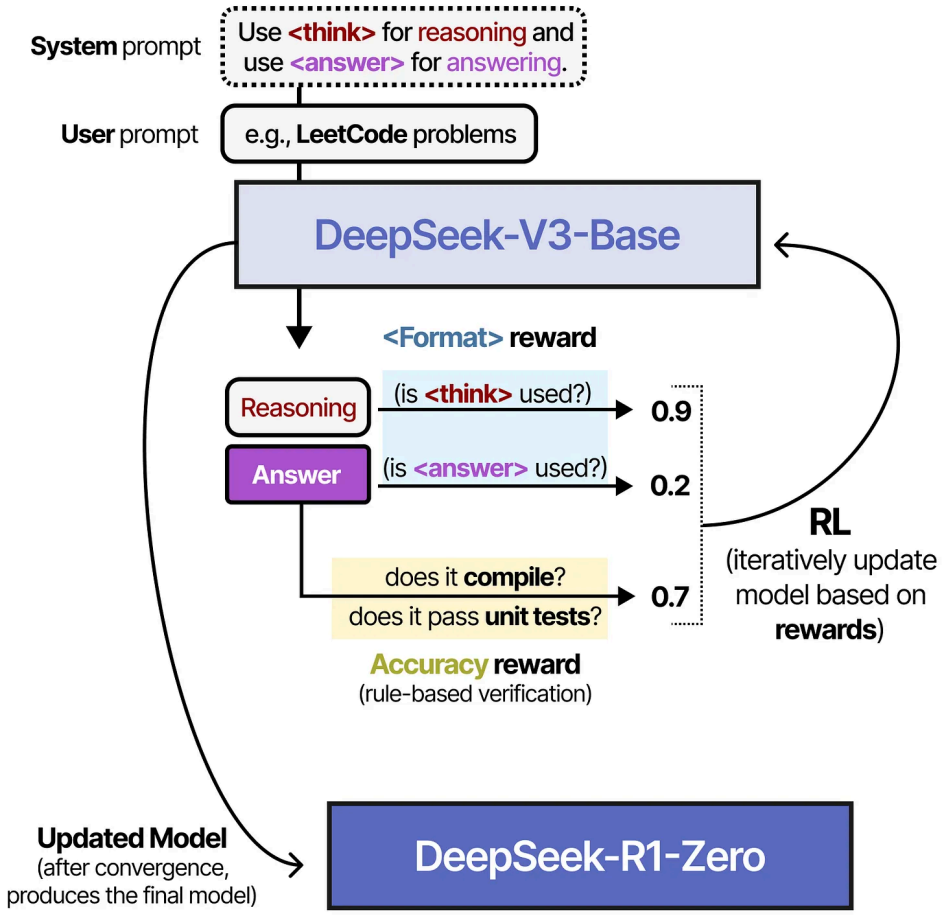
## RLVR con GRPO - Calcular la pérdida

$$L_{PG} = -\frac{1}{N}\sum_{i=1}^{N} A_i \sum_{t=1}^{T_i} \log p_w(y_t^i \mid y_{<t}^i, x^i)$$

# RLVR con GRPO - Ajustar los pesos (¡igual que siempre!)



# (2) Reinforcement Learning: DeepSeek-R1 Zero



# (2) Reinforcement Learning: DeepSeek-R1 Zero

| |
|---|
| Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to |
| Response: <think><br>To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$<br>$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$.<br>Rearrange to isolate the inner square root term:<br>$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$<br>$\ldots$<br>Wait, wait. Wait. That's an aha moment I can flag here.<br>Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$<br>We started with the equation:<br>$\sqrt{a - \sqrt{a + x}} = x$<br>First, let's square both sides:<br>$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$<br>Next, I could square both sides again, treating the equation: $\cdots$<br>$\ldots$ |

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

# (2) Reinforcement Learning: DeepSeek-R1 Zero

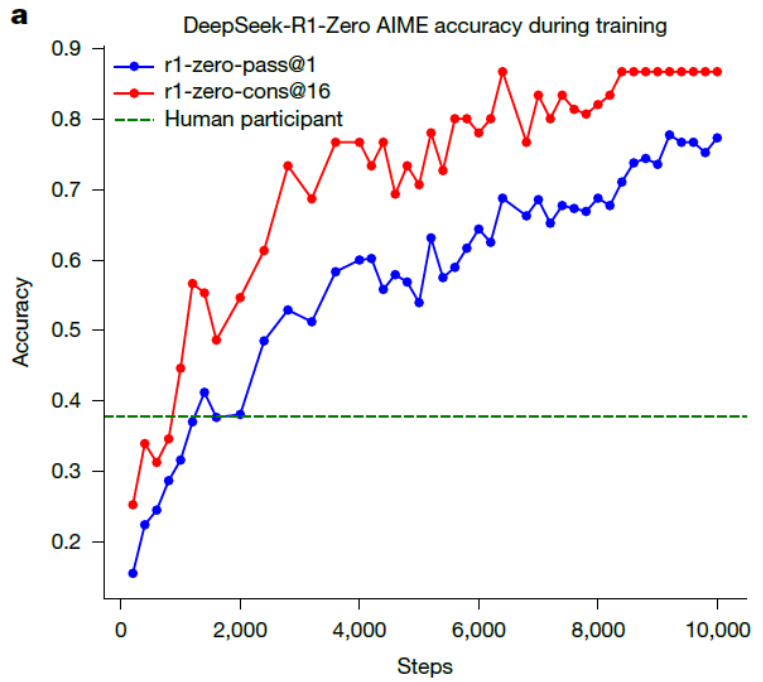
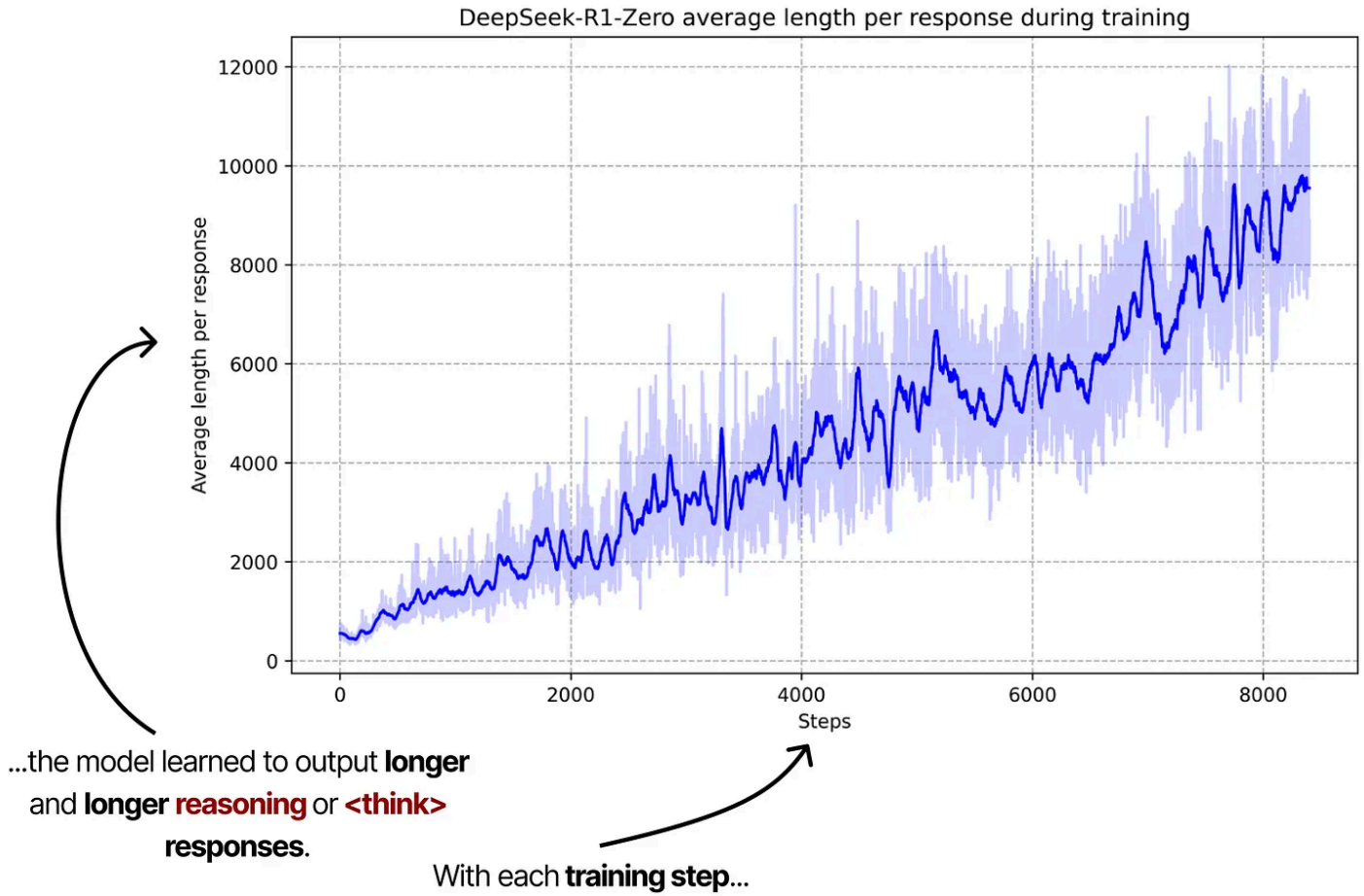
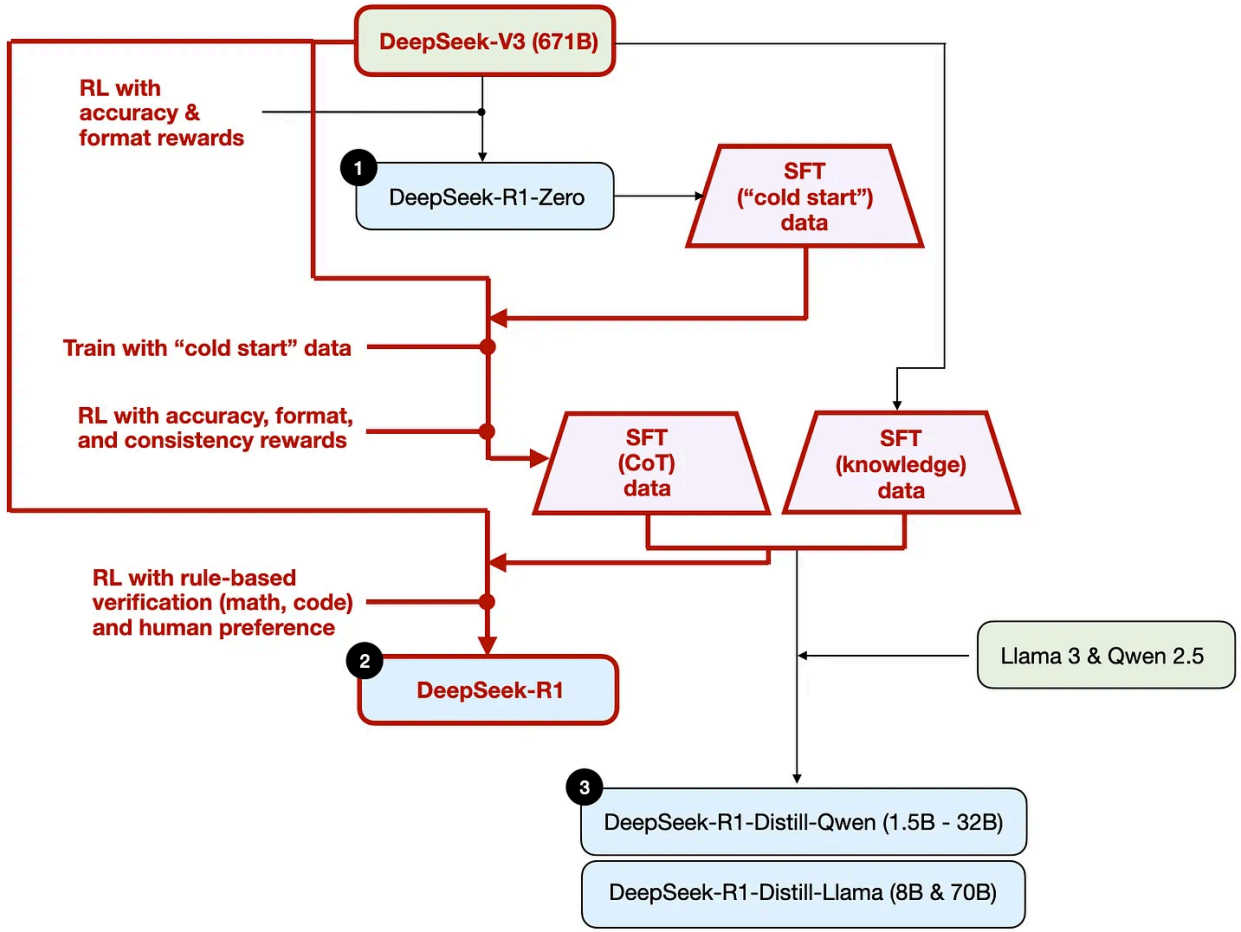



**Fig. 1 | Accuracy and output length of DeepSeek-R1-Zero throughout the training process. a**, AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Extended Data Table 1. pass@1 and cons@16 are described in Supplementary Information, section 4.1. The baseline is the average score achieved by human

# (2) Reinforcement Learning: DeepSeek-R1 Zero

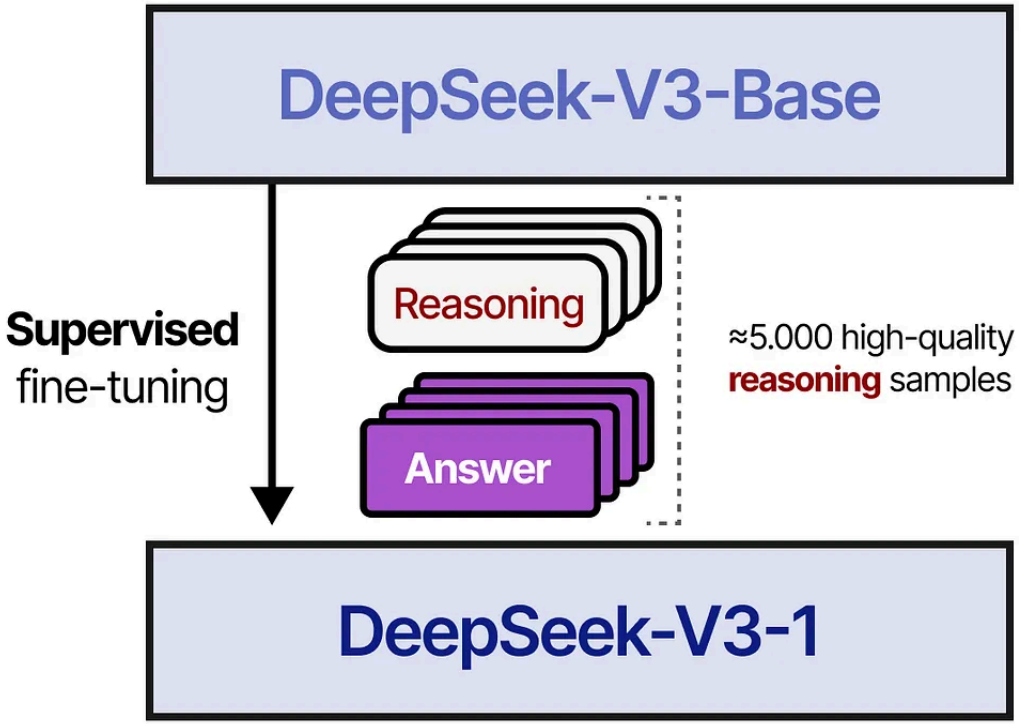


# (3) SFT+RL: Deepseek-R1



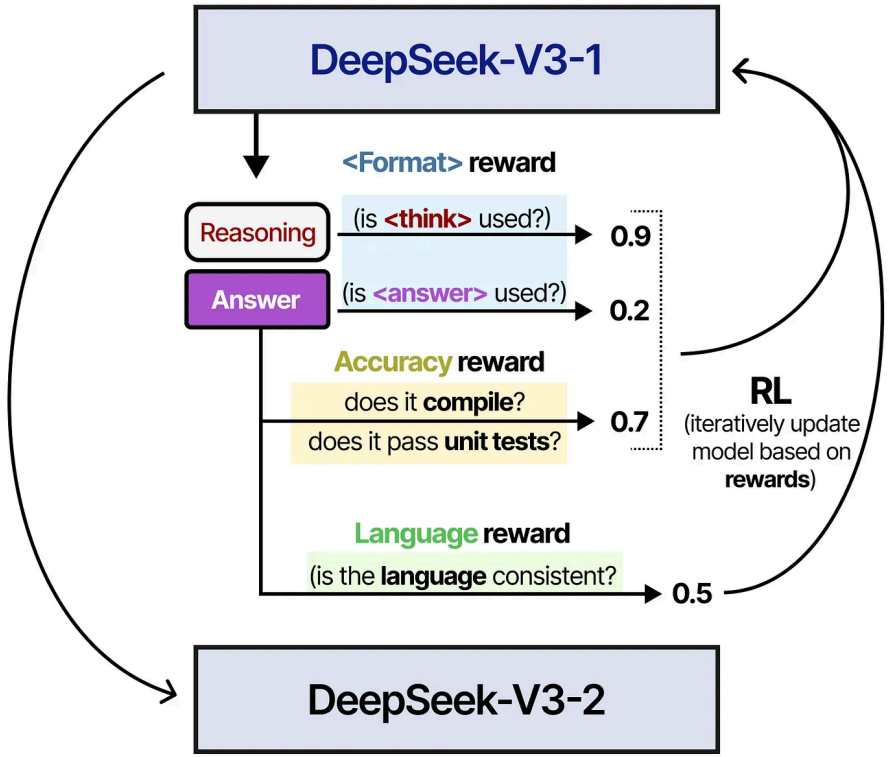
# (3) SFT+RL: Deepseek-R1

- Cold start: 5000 ejemplos de razonamiento



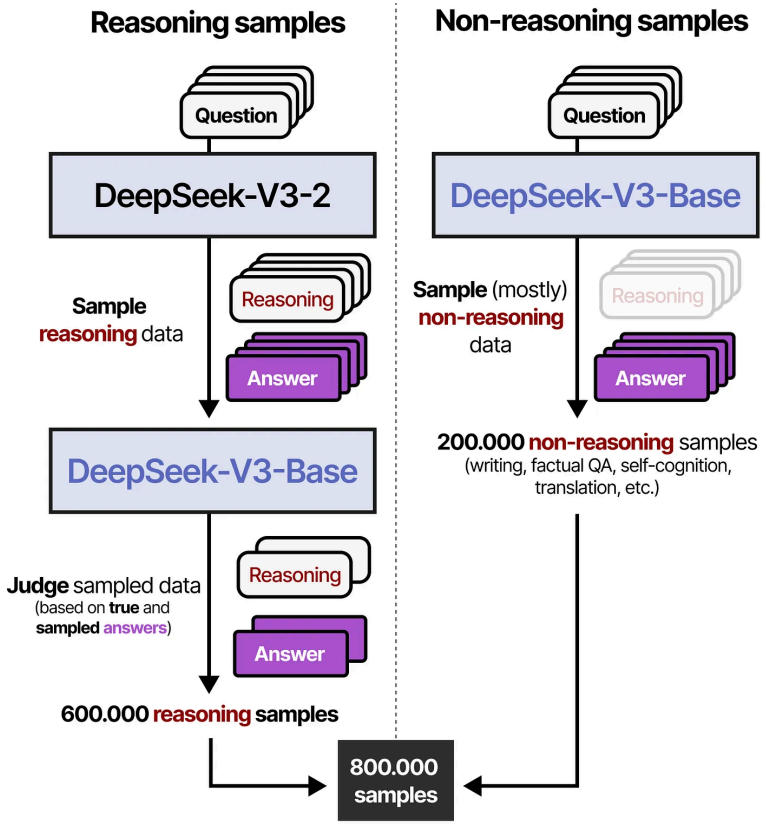
# (3) SFT+RL: Deepseek-R1

- Más RL, pero con un "language reward", para mejorar la legibilidad



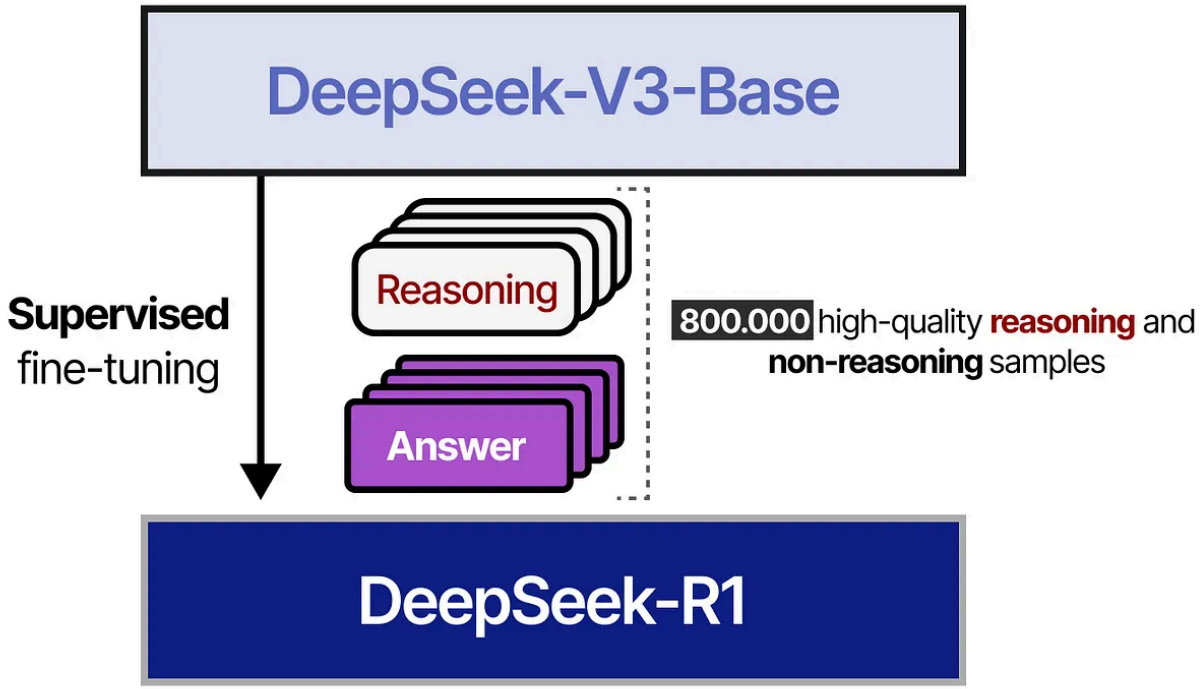
# (3) SFT+RL: Deepseek-R1

- Generación de datos sintéticos utilizando Deepseek-V3-2



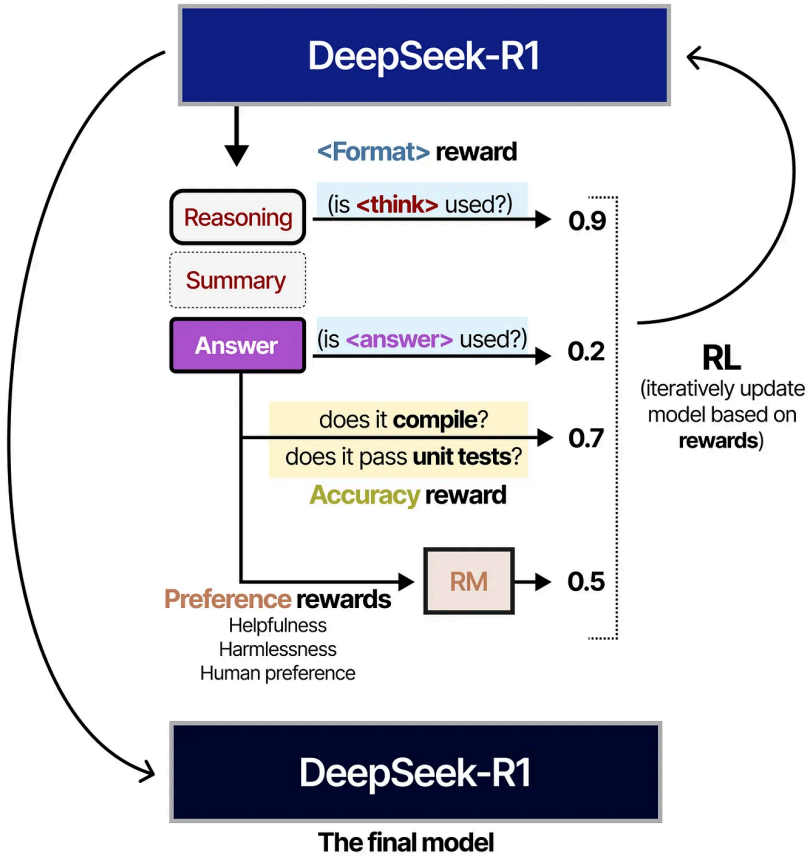
# (3) SFT+RL: Deepseek-R1

- Supervised Fine Tuning!



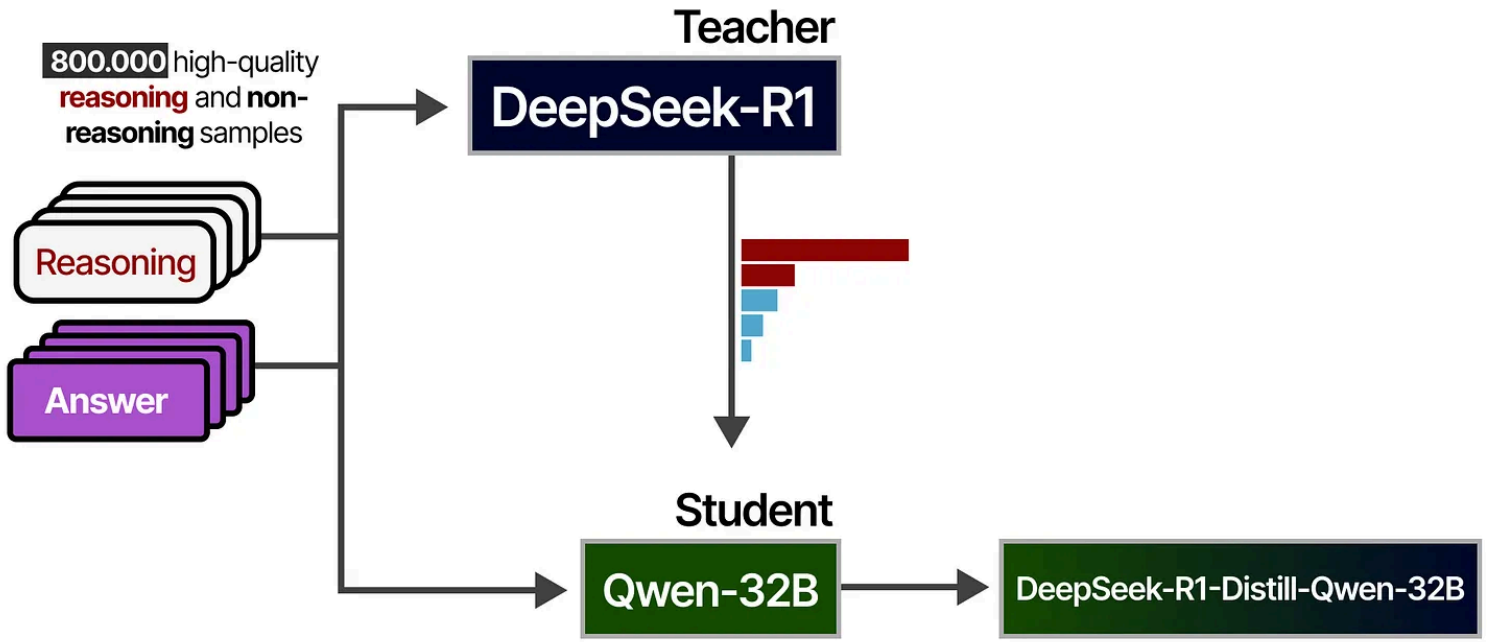
# (3) SFT+RL: Deepseek-R1

- Alignment y más RL



# (4) Destilación

- Usamos DeepSeek-R1 para "enseñarle" a Qwen-32B a razonar.
- Idea: el estudiante cambia la distribución del next-token para que se parezca al del maestro o directamente utiliza ejemplos generados por el maestro para hacer fine tuning



# (4) Resultados

| Model | AIME 2024 | | MATH-500 | GPQA Diamond | LiveCode Bench | CodeForces |
|---|---|---|---|---|---|---|
| | pass@1 | cons@64 | pass@1 | pass@1 | pass@1 | rating |
| GPT-4o-0513 | 9.3 | 13.4 | 74.6 | 49.9 | 32.9 | 759 |
| Claude-3.5-Sonnet-1022 | 16.0 | 26.7 | 78.3 | 65.0 | 38.9 | 717 |
| OpenAI-o1-mini | 63.6 | 80.0 | 90.0 | 60.0 | 53.8 | **1820** |
| QwQ-32B-Preview | 50.0 | 60.0 | 90.6 | 54.5 | 41.9 | 1316 |
| DeepSeek-R1-Distill-Qwen-1.5B | 28.9 | 52.7 | 83.9 | 33.8 | 16.9 | 954 |
| DeepSeek-R1-Distill-Qwen-7B | 55.5 | 83.3 | 92.8 | 49.1 | 37.6 | 1189 |
| DeepSeek-R1-Distill-Qwen-14B | 69.7 | 80.0 | 93.9 | 59.1 | 53.1 | 1481 |
| DeepSeek-R1-Distill-Qwen-32B | **72.6** | 83.3 | 94.3 | 62.1 | 57.2 | 1691 |
| DeepSeek-R1-Distill-Llama-8B | 50.4 | 80.0 | 89.1 | 49.0 | 39.6 | 1205 |
| DeepSeek-R1-Distill-Llama-70B | 70.0 | **86.7** | **94.5** | **65.2** | **57.5** | 1633 |
| **DeepSeek-R1-Zero** | **71.0** | | **95.9** | **73.3** | **50.0** | **1444** |
| **DeepSeek-R1** | **79.8** | | **97.3** | **71.5** | **65.9** | **2029** |

# ¿Podemos mejorar un modelo usando Prolog?

- Queremos enseñarle al modelo a usar Prolog
- Problemas matemáticos, con su respuesta
- Linea base: Supervised Fine Tuning
- Alternativa: GRPO!

*Training language models to use prolog as a tool (Mellgren et al., 2026*

.

# Ejemplos

```
{
    "instruction": "Please generate a piece of Prolog code to solve the given math problem.",
    "input": "Natalia sold clips to 48 of her friends in April,
    and then she sold half as many clips in May.
    How many clips did Natalia sell
    altogether in April and May?",

    "output": ":- use_module(library(clpq)).
    sell_clips(natalia, april, 48).

    solve(Total_amount) :-
    sell_clips(natalia, april, April_amount),
    {May_amount = April_amount / 2},
    {Total_amount = May_amount + April_amount}."
}
```

# Prompt

```
You are a specialized Prolog code-generating assistant.
Your task is to solve math problems by providing a structured answer in two clearly defined sections:

1. <reasoning>
- Provide a clear, concise step-by-step explanation of how you arrive at the solution.

2. <answer>
- Provide executable Prolog code using constraint logic programming to compute the numeric answer.
- Always start with: ':- use_module(library(clpq)).'
<...>
Use this XML format strictly:
<reasoning>
(Your step-by-step reasoning here)
</reasoning>
<answer>
:- use_module(library(clpq)).

(Any predicates/constants defined here)

solve(X) :-
(Intermediate computations using curly braces)
{X = final constraint logic}.
</answer>
```

## Rewards

- Corrección
- Sintaxis Prolog
- Formato de Respuesta
- XML Tags
- ...

## Inferencia

- Prolog-as-output
  - Single-Try
  - Multiple-Try
- Prolog-as-a-tool (Agentes)

# Resultados

| Model | MMLU-Stem | MMLU-Pro |
|---|---|---|
| **Our GRPO-trained Qwen2.5 models with 3B parameters and zero-shot evaluation** | | |
| SP-Struct-RWD1-full Single-Try | 50.93 | 21.33 |
| SP-Struct-RWD1-full Multiple-Try | 53.60 | 26.67 |
| SP-Struct-RWD1-full Agentic-Internal | 56.80 | **30.67** |
| SP-Struct-RWD1-full Agentic-Independent | **58.13** | 29.33 |
| **Baselines from a weight class above (7B) and 5-shot evaluation** | | |
| DeepSeekMath Base-7B (few-shot) | **56.50** | – |
| Mistral 7B (5-shot) | 51.10 | 30.90 |
| Gemma 7B (5-shot) | – | **33.70** |

# PgProlog

## Un dataset de problemas

(Aníbal Presa, Agustín Amegeiras, Bruno de Simone)

# Links y Referencias

- Understanding Reasoning LLMs - Sebastian Raschka - https://magazine.sebastianraschka.com/p/understanding-reasoning-llms
- A Visual Guide to Reasoning LLMs - Maarten Grootendorst - https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-reasoning-llms
- Build a Reasoning Model (From Scratch) - Sebastian Raschka
- Understanding Reasoning Models & Test-Time Compute: Insights from DeepSeek-R1 - ChiChieh Huang - https://medium.com/@cch.chichieh/understanding-reasoning-models-test-time-compute-insights-from-deepseek-r1
- Papers citados en las slides

(Las imágenes de la presentación son tomadas de las dos primeras referencias)

# ¿Preguntas?

QAuy

The success of pure RL depends on reliable reward signals. In this study, we ensure reward reliability through a reasoning-domain rule-based reward model. However, such dependable reward models are difficult to construct for certain tasks, such as writing. If the reward signal is assigned by a model instead of predefined rules, it becomes more susceptible to exploitation as training progresses, which means that the policy model may find shortcuts to hack the reward model. Consequently, for complex tasks that cannot be effectively evaluated by a reliable reward model, scaling up pure RL methods remains an open challenge.

In this work, for tasks that cannot obtain a reliable signal, DeepSeek-R1 uses human annotation to create supervised data and only conducts RL for hundreds of steps. We hope that, in the future, a robust reward model can be obtained to address such issues.

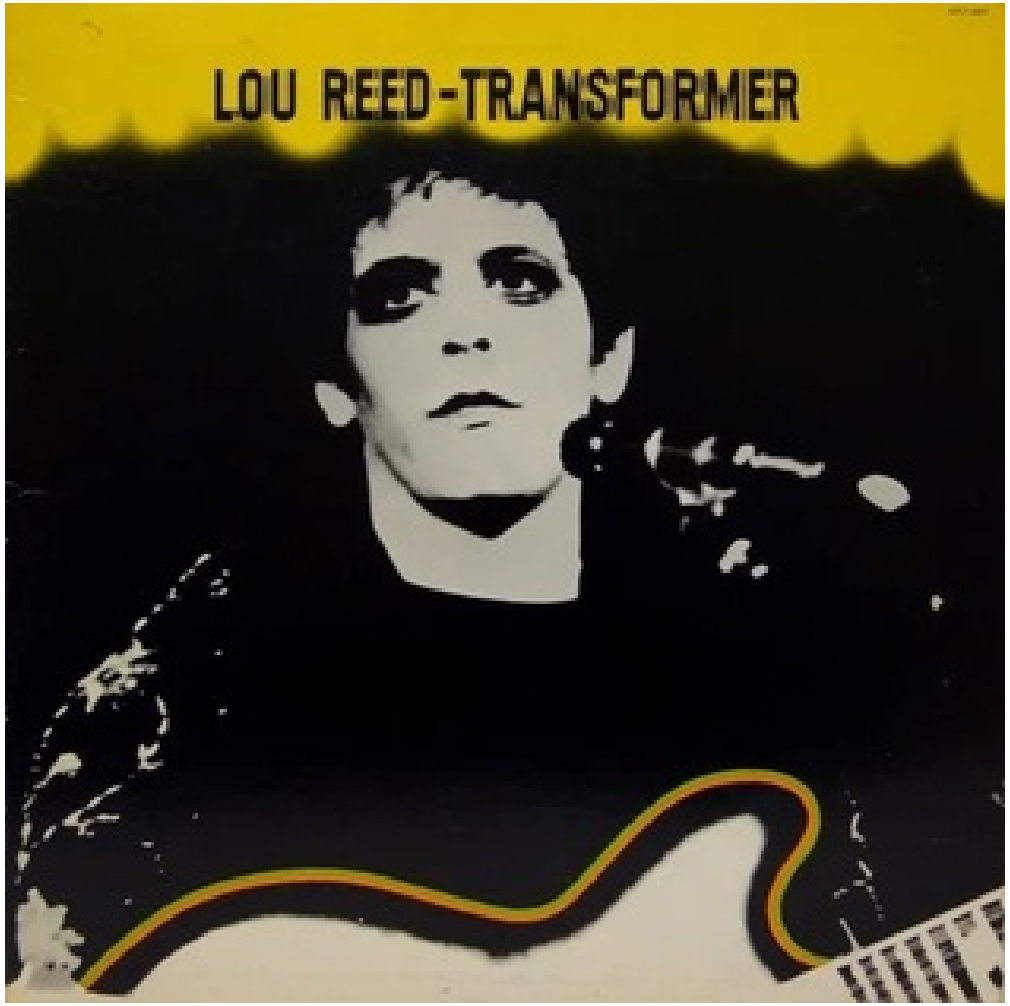
*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement DeepSeek AI (2025)*

*With AI now, we are able to write new programs that we could never hope to write by hand before. We do it by specifying objectives (e.g. classification accuracy, reward functions), and we search the program space via gradient descent to find neural networks that work well against that objective. This is my Software 2.0 blog post from a while ago. In this new programming paradigm then, the new most predictive feature to look at is verifiability. If a task/job is verifiable, then it is optimizable directly or via reinforcement learning, and a neural net can be trained to work extremely well. It's about to what extent an AI can "practice" something.*

*Andrej Karpathy, https://x.com/karpathy/status/1990116666194456651?s=20*

*The more a task/job is verifiable, the more amenable it is to automation in the new programming paradigm. If it is not verifiable, it has to fall out from neural net magic of generalization fingers crossed, or via weaker means like imitation. This is what's driving the "jagged" frontier of progress in LLMs. Tasks that are verifiable progress rapidly, including possibly beyond the ability of top experts (e.g. math, code, amount of time spent watching videos, anything that looks like puzzles with correct answers), while many others lag by comparison (creative, strategic, tasks that combine real-world knowledge, state, context and common sense).*

*Andrej Karpathy, https://x.com/karpathy/status/1990116666194456651?s=20*


LOU REED - TRANSFORMER